FairyWREN: A Sustainable Cache for Emerging Write-Read-Erase Flash Interfaces

SARA MCALLISTER, Computer Science, Carnegie Mellon University, Pittsburgh, United States
YUCONG WANG, Computer Science, Carnegie Mellon University, Pittsburgh, United States and Salesforce Inc, San Francisco, United States
BENJAMIN BERG, Computer Science, The University of North Carolina at Chapel Hill, Chapel Hill, United States
DANIEL S. BERGER, Microsoft Research, Redmond, United States and University of Washington, Seattle, United States
NATHAN BECKMANN, Computer Science, Carnegie Mellon University, Pittsburgh, United States
GEORGE AMVROSIADIS, Carnegie Mellon University College of Engineering, Pittsburgh, United States
GREGORY R. GANGER, Electrical and Computer Engineering, Carnegie Mellon University, Pittsburgh, United States

Datacenters need to reduce embodied carbon emissions, particularly for flash, which accounts for 40% of embodied carbon in servers. However, decreasing flash's embodied emissions is challenging due to flash's limited write endurance, which more than halves with each generation of denser flash. Reducing embodied emissions requires extending flash lifetime, stressing its limited write endurance even further. The legacy Logical Block-Addressable Device (LBAD) interface exacerbates the problem by forcing devices to perform garbage collection, leading to even more writes.

Flash-based caches in particular write frequently, limiting the lifetimes and densities of the devices they use. These flash caches illustrate the need to break away from LBAD and switch to the new Write-Read-Erase iNterfaces (WREN) now coming to market. WREN affords applications control over data placement and garbage collection. We present FairyWren,[1] a flash cache designed for WREN. FairyWren reduces writes by

[1]Fairywrens () are vibrant birds native to Australia. Common varieties include Superb Fairywrens, Splendid Fairywrens, and Lovely Fairywrens.

Yucong Wang work performed while at CMU.
Sara McAllister is supported by a NDSEG Fellowship and a Siebel Scholarship. This work is also supported by NSF CIF-241615 and NSF IIS-233630 grants.
Authors' Contact Information: Sara McAllister, Computer Science, Carnegie Mellon University, Pittsburgh, Pennsylvania, United States; e-mail: sjmcalli@cs.cmu.edu; Yucong Wang, Computer Science, Carnegie Mellon University, Pittsburgh, Pennsylvania, United States and Salesforce Inc, San Francisco, California, United States; e-mail: yucongw@andrew.cmu.edu; Benjamin Berg, Computer Science, The University of North Carolina at Chapel Hill, Chapel Hill, North Carolina, United States; e-mail: bsberg@andrew.cmu.edu; Daniel S. Berger, Microsoft Research, Redmond, Washington, United States and University of Washington, Seattle, Washington, United States; e-mail: Daniel.Berger@microsoft.com; Nathan Beckmann, Computer Science, Carnegie Mellon University, Pittsburgh, Pennsylvania, United States; e-mail: beckmann@cmu.edu; George Amvrosiadis, Carnegie Mellon University College of Engineering, Pittsburgh, Pennsylvania, United States; e-mail: gamvrosi@andrew.cmu.edu; Gregory R. Ganger, Electrical and Computer Engineering, Carnegie Mellon University, Pittsburgh, Pennsylvania, United States; e-mail: ganger@andrew.cmu.edu.

ACM 1553-3077/2025/11-ART26
https://doi.org/10.1145/3718390

co-designing caching policies and flash garbage collection. FairyWren provides a 12.5× write reduction over state-of-the-art LBAD caches. This decrease in writes allows flash devices to last longer, decreasing flash cost by 35% and flash carbon emissions by 33%.

CCS Concepts: • **Information systems → Flash memory**; **Storage management;**

Additional Key Words and Phrases: Flash-based caches, flash interfaces

ACM Reference Format:
Sara McAllister, Yucong Wang, Benjamin Berg, Daniel S. Berger, Nathan Beckmann, George Amvrosiadis, and Gregory R. Ganger. 2025. FairyWREN: A Sustainable Cache for Emerging Write-Read-Erase Flash Interfaces. *ACM Trans. Storage* 21, 4, Article 26 (November 2025), 34 pages. https://doi.org/10.1145/3718390

1 Introduction

Datacenter carbon emissions are a topic of growing concern. At current emission rates, datacenters' share of global emissions are projected to rise to 20% by 2038 [48] and 33% by 2050 [53]. In the next few decades, many companies—including Amazon [1], Google [2], Meta [11], and Microsoft [71]—are looking to achieve Net Zero, i.e., greenhouse gas emissions close to zero. To achieve this goal, many datacenters are adopting renewable energy sources such as solar and wind [11, 39, 64, 71]. Google, AWS, and Microsoft are expected to complete their transition to renewable energy by 2030 [30, 49, 59]. However, this switch in energy source does not reduce datacenters' *embodied emissions*, the emissions produced by the manufacture, transport, and disposal of datacenter components. Embodied emissions will account for more than 80% of datacenter emissions once datacenters move to renewable energy [39].

Embodied emissions are produced by one-time lifecycle events. Datacenters can reduce these emissions by (i) replacing hardware with less carbon-intensive alternatives and (ii) extending the lifetime of components to amortize embodied emissions over a longer period. Recent work has studied embodied emissions in processor design [24, 38, 39, 85], but considerably less attention has been paid to memory and storage, even though they constitute 46% and 40% of server emissions, respectively [64]. It is therefore crucial to both move from carbon-intensive technologies like DRAM to flash, which has 12× less embodied carbon per bit [38], *and* to extend flash lifetimes to amortize flash's embodied carbon.

However, flash introduces a new challenge: *limited write endurance*. A flash device can only be written a limited number of times before it wears out. Each new generation of flash has lower write endurance as a result of manufacturers packing more bits into each cell. This packing, however, does improve sustainability by storing more capacity in the same silicon (i.e., less carbon per bit). To realize the benefits of denser flash, applications must write to flash much less frequently. The write-rate budgets that applications must operate under to achieve longer lifetimes are tiny: to achieve a six-year lifetime on a 2-TB **quad-level cells (QLC)** drive, the application can write only 14 MB/s, or 0.09% of available write bandwidth (Section 2).

Reducing carbon from caching. Hence, write-intensive flash applications present a major challenge in reducing overall datacenter emissions. This paper focuses on reducing carbon from flash caching, an increasingly popular use of flash in the datacenter [3, 16, 21, 22, 35, 36, 83]. We aim to demonstrate, through caching, how to *leverage emerging flash interfaces* to reduce writes, in particular by *re-purposing garbage collection to do useful work*.

Caching is fundamentally write intensive, as new objects must be frequently admitted to maintain hit rates [15, 18]. Datacenter caches also store many small objects [16, 67], which is particularly problematic, because flash can only be written at a coarse granularity. Because of this mismatch,

admitting small objects to the cache can lead to significant *write amplification*: i.e., more bytes are written to the underlying flash device than requested by the application.

Most current flash devices are **Logical Block-Addressable Devices (LBAD)** that present the same block device abstraction used by hard disks. This abstraction hides significant details about how SSDs work. In particular, while the interface allows reading and writing 4-KB blocks, the underlying flash device can only erase large (MB to GB) regions. To implement the LBAD interface, the flash firmware performs garbage collection, copying blocks of valid data and erasing entire regions to make room for new writes. Current flash caches, such as the research state-of-the-art Kangaroo [67, 68], have a limited ability to optimize these internal writes, which can amplify the total bytes written by 2× to 10× [67].

Opportunity: Write-Read-Erase iNterfaces. New flash SSD interfaces, such as **Zoned Namespaces (ZNS)** [19] and **Flexible Data Placement (FDP)** [66], allow closer integration of host-level software and flash management. The key difference between these interfaces and LBAD is that these interfaces include `Erase` as a first-order operation, allowing the cache to control garbage collection. We use the name **Write-Read-Erase iNterfaces (WREN)** to collectively refer to such interfaces, and we describe the necessary and sufficient operations for flash caches to minimize write rate. However, we also show that merely porting existing flash caches to WREN does not reduce flash writes. *Flash caches must be re-designed to leverage the additional control provided by WREN.*

Our solution: FairyWren. We design and implement FairyWren, a flash cache that harnesses WREN to reduce writes. The main insight in FairyWren is that every flash write, whether from the application *or from garbage collection*, is an opportunity to admit objects to the cache. When flash is written during garbage collection, FairyWren can admit objects "for free." This idea cannot be realized on LBAD, since these devices offer no control over garbage collection. FairyWren uses the features of WREN to perform a "nest packing" algorithm on *every* write, *unifying cache admission and garbage collection in a single algorithm.* FairyWren also leverages WREN to enable large–small object separation and hot–cold set-partitioning, further reducing writes.

Summary of results. We find that, without major changes to flash interfaces and cache designs, deploying denser flash will not reduce the carbon emissions of flash caches. *For current caching systems, the reduced write endurance of denser flash outweighs the gains in density.* Only by changing the flash interface and optimizing the cache to this new interface can we realize the significant emissions savings of denser flash.

To illustrate this point, we implement FairyWren as a flash cache module within CacheLib [16]. We evaluate FairyWren on production traces from Meta and Twitter using both simulation and a real ZNS SSD. *FairyWren reduces flash writes by 12.5*× vs. the research state of the art. By enabling caching on denser flash, *FairyWren reduces flash's carbon emissions by 33%* vs. the research state of the art (Figure 1). FairyWren performs close to an idealized, minimum-write cache on both carbon emissions and cost.

Contributions. This paper contributes the following:

— *Flash trends (Section 2):* By studying flash trends, we identify opportunities for more sustainable flash caching as well as challenges that prevent current flash caches from realizing these benefits (Section 3).

— *Critical elements of flash interfaces (Section 4):* We identify the `Erase` operation and control over garbage collection as the essential features of emerging flash interfaces. We describe tradeoffs and fundamental constraints of flash interfaces, showing that some features are, contrary to prior work, unhelpful for caching.

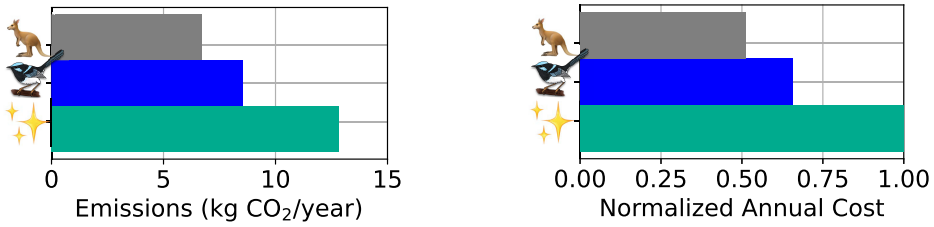


Fig. 1. Carbon emissions and cost for flash in Kangaroo ( ), FairyWren ( ), and "minimum writes" ( )—an idealized cache with no extra writes—over a 6-year lifetime for a production Twitter trace and a target 30% miss ratio. Compared to Kangaroo, FairyWren reduces carbon emissions by 33% and cost by 35%.

— *Analysis of erase granularity in WREN (Section 4.4):* We analyze the effect of `Erase` operation's granularity in WREN, bridging the theoretical and systems understanding of its impact on write amplification.
— *FairyWren (Section 5):* FairyWren's key insight is to leverage emerging flash interfaces to unify garbage collection and cache admission as one operation, greatly reducing overall flash writes. FairyWren further reduces writes by partitioning objects by size and popularity (hot vs. cold).
— *Model of caching's carbon emissions (Section 6.2):* We develop a model to analyze carbon emissions from flash caching — incorporating both write rate and cache capacity to determine overall flash emissions.
— *Analysis of cache write amplification and its impact on emissions (Sections 6.3–6.7):* We show that FairyWren's write reduction allows flash caches to improve sustainability using denser flash for longer lifetimes, without increasing the cache's miss ratio.

2 Opportunities in Flash Caching

Flash is an increasingly attractive option for caching [16, 21, 22, 35, 57, 67, 68, 83]. In this section, we discuss how trends in the design of flash devices present growing opportunities to reduce the cost and carbon emissions of caching.

Opportunity 1: Flash is less carbon intensive than DRAM, so caches are more sustainable with less DRAM.

DRAM often makes up 40% to 50% of server cost [58, 79, 82] and is no longer scaling (Figure 2). DRAM also has a large embodied carbon footprint and has large operational emissions due to requiring up to half of system power [38].

Flash is cheaper per-bit, embodies 12× less carbon, and requires less power per-bit than DRAM [38]. Thus, datacenters should use flash over DRAM whenever possible [37], even for traditionally DRAM workloads, such as caching [16, 35, 67, 68] or machine learning [95].

Opportunity 2: Flash caches should use denser flash where possible to reduce emissions.

Flash is becoming denser, moving from **single-level cells (SLC)**, which store 1 bit/cell, to **tri-level cells (TLC)**, which store 3 bits/cell. Flash SSDs will soon use QLC and **penta-level cells (PLC)** [73]. Denser flash is cheaper; e.g., PLC is forecast to be 40% cheaper per-bit than TLC [9]. Denser flash also reduces carbon emissions, since more bits are packed onto roughly the same silicon.

Opportunity 3: Lengthening device lifetime is an effective way to improve datacenter sustainability.

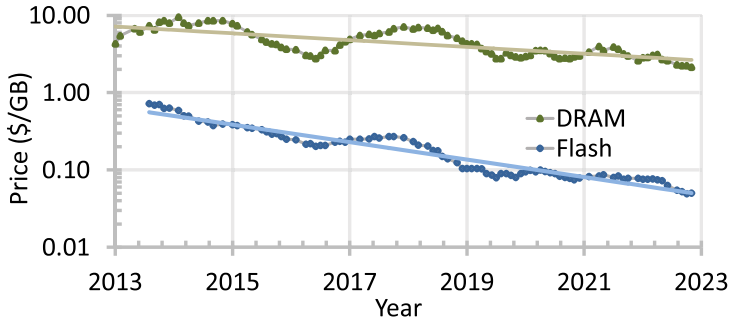


Fig. 2. Cost for flash and DRAM over the past 10 years [4, 6]. Flash prices have decreased over 14×, while DRAM prices have only decreased by ≈2×.

Traditionally, datacenter hardware replacement cycles have been around 3 years [64] due to the rate of improvement in hardware performance and energy efficiency. Today, datacenters deploy devices for longer. Longer replacement cycles have become common due to their cost advantages and the slowing of Moore's Law. For example, Microsoft Azure increased the depreciable lifetime of servers from 4 to 6 years [42, 65], and Meta recently started planning for servers to last 5.5 years [12]. Additionally, hyperscalers are finding that servers do not fail quickly: Failure rates at Azure have little evidence of increasing before 8 years [17, 64].

Moving to longer lifetimes amortizes both cost and embodied carbon. As datacenters shift to renewable energy, they are rapidly reducing operational carbon. As a result, embodied carbon now dominates datacenter carbon emissions [12, 38, 39, 84]. The major challenge, though, is how to extend *flash* lifetime, given its limited write endurance.

3 Challenges in Flash Caching

Flash SSDs have limited write endurance and are warrantied only for a stated write budget [10]. Exceeding this write budget can cause the device to fail. Hence, while flash caching presents carbon-saving opportunities (Section 2), caches must severely limit the amount they write. Here, we discuss the challenges of flash caching in detail and describe how current systems fail to address these challenges.

3.1 Wherefore Device Write Amplification?

Flash devices cannot write new values without first erasing a large region of the device. To support random writes, devices must read all live data in a region, erase the region, and then write the live data back to the drive along with any new data. As a result, flash SSDs perform more writes than requested by the application. The **device-level write amplification (DLWA)** [23, 35, 41, 54, 57, 62, 83] captures this relative increase in bytes actually written to flash vs. bytes written by an application. (If an SSD writes 3 GB to serve 1 GB of application writes, then DLWA is 3×.) DLWA can be large: A factor of 2× to 10× is common [67]. DLWA causes write-intensive applications to quickly wear out flash devices, increasing their replacement frequency and embodied emissions over time.

DLWA is primarily caused by the physical limitations of flash storage. Flash devices are organized in a physical hierarchy (Figure 3). The smallest unit is the *page*, usually 4 KB. Flash can be written at page granularity, but a page must be erased before it can be rewritten. To avoid electrical interference during erasure, pages are grouped into *flash blocks* [13, 19, 20, 41, 63]. A flash block is the minimum erase size. In practice, however, flash drives stripe writes across blocks to

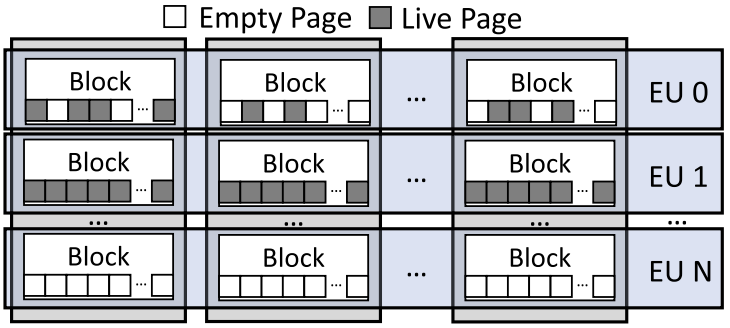


Fig. 3. The internal arrangement of flash devices into planes, blocks, pages, and EUs. Each EU has blocks in multiple pages. EU 0 is a partially full, EU 1 is entirely full, and EU N has just been erased.

improve bandwidth and error correction. Striping increases the effective **erase unit (EU)** size to gigabytes [19].

The mismatch between the granularity of writes and erases is the root cause of DLWA. To maintain the 4-KB read/write block interface, flash devices **garbage collect (GC)**, moving live pages from partially empty EUs (such as EU 0 in Figure 3) to a writable EU (such as EU N) before erasing the EU and freeing dead pages. The less the available capacity on the device, the more frequently it has to GC, introducing a tradeoff between flash utilization and flash writes.

One might hope that technological advances would decrease EU sizes, closing the gap between write and erase granularities. However, *flash EU sizes have gotten larger as flash has gotten denser.* Effective block sizes on an SLC flash device were 128 KB [86], MLC and TLC flash devices are around 20 MB [81], and QLC devices will be 48 MB [80]. Striping these blocks with hundreds of 3D-stacked layers [80] results EUs in the gigabyte range [19, 69].

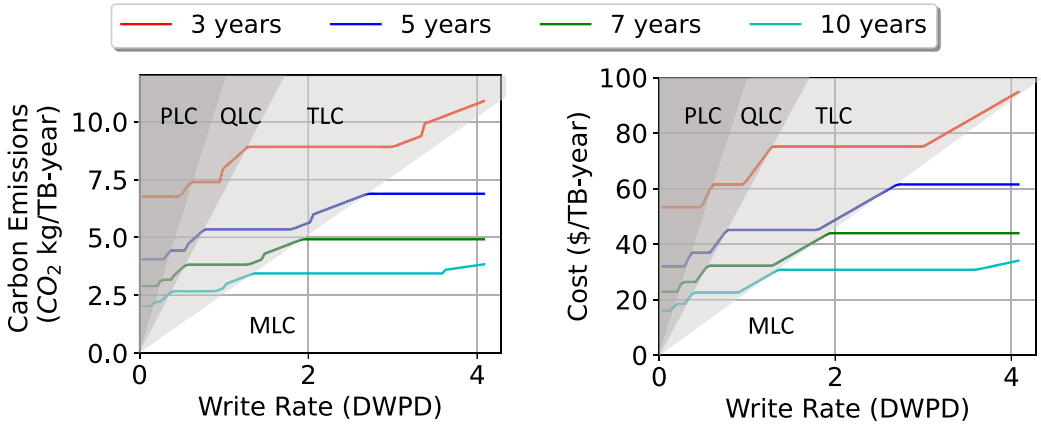
Lesson for flash caches: Write amplification is caused by the size mismatch between writes and erases in flash. This mismatch will keep increasing.

3.2 Denser Flash Has Lower Write Endurance

As flash becomes denser, its write endurance drops significantly. For example, while PLC flash is up to 40% denser than TLC, PLC is forecast to have only *16%* of TLC's writes [9]. Additionally, because denser flash has to differentiate between more voltage levels, even small voltage changes can make data unreadable. TLC uses two-phase writes and more frequent refresh to prevent data loss [70]. Two-phase writes require the device to have enough RAM and capacitance to remember all in-flight writes, limiting the number of EUs that can be "active" (i.e., writable) at any point in time, often to less than 10. Writing to more EUs than this requires closing an active EU, incurring more internal device writes.

Figure 4 models how write rate affects both emissions and cost when varying lifetimes and flash density. Each line shows a device of a different lifetime, and shaded regions show which flash density is best for a given write rate. The model calculates how much capacity must be provisioned for each technology to achieve the desired lifetime at a given write rate. For example, a device lasting 7 years (green) has lower annualized carbon emissions than one lasting 3 or 5 years, and it should use dense flash (e.g., TLC) only at write rates below two device-writes-per-day.

Lesson for flash caches: Device lifetime is the most important factor in reducing carbon emissions. Moreover, denser flash can improve sustainability, but only if flash write rate is very small—much less than one device-write per day.



(a) Carbon emissions. *(b) Cost.*

Fig. 4. The annual carbon emissions and cost of flash depending on the required average write rate and desired lifetime.

Table 1. Comparison of FairyWREN vs. Prior Cache Designs

	Flash caches should minimize ...			
	Unused flash	**DRAM**	**ALWA**	**DLWA**
Key-value stores	✗	✓	✓	✓
Log-structured caches	✓	✗	✓	✓
Set-associative caches	✗	✓	✗	✗
Kangaroo [67]	✓	✓	✓	✗
FairyWREN	✓	✓	✓	✓

FairyWREN is the only design to minimize all important overheads.

3.3 Shortcomings of Existing Solutions

To limit embodied emissions, sustainable flash caches must minimize (i) idle flash space, which incurs emissions for no benefit; (ii) DRAM usage for object metadata, which can add up to tens of GBs [35, 67]; and (iii) flash write rates, which wear out the device, reducing lifetime. No prior flash-cache design meets these criteria (Table 1). In particular, although caches must admit new objects to maintain hit rates, flash caches must be designed to minimize application- and device-level write amplification to extend device lifetime.

Flash caches ≠ DRAM caches. Both flash caches and DRAM caches try to reduce misses, but flash caches must also contend with flash's limited write endurance, leading to much different designs. Flash caches are designed to achieve low end-to-end write amplification, i.e., the product of **application-level write amplification (ALWA)** (e.g., from having to write 4 KB to flash to admit a 100-B object) and DLWA.

Flash caches ≠ key-value stores. KV stores [5, 7, 33, 55, 60, 75, 90] support a similar read-write interface as caches and likewise minimize flash writes and DRAM overhead. However, flash caches have significantly different design goals.

The main difference is that `delete` operations are uncommon in KV stores but very frequent in caches. Caches frequently evict objects and must reclaim space immediately to admit new objects [67]. Most KV stores do not support deleting objects quickly enough to implement cache eviction

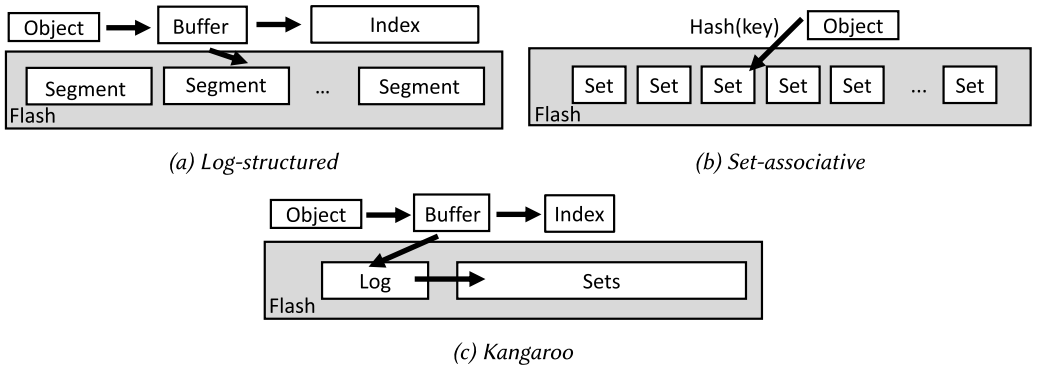


Fig. 5. Designs of prior flash caches: (a) log-structured caches write objects segments to flash sequentially, (b) set-associative cache write objects to a set based on the key's hash, and (c) Kangaroo is a hierarchical design that combines a log-structured and a set-associative cache.

policies. Specifically, standard KV store data structures like LSM trees [5, 7, 31, 32, 60, 75, 90] will not work well for caching unless the KV store is massively overprovisioned, often by more than 2× the cache capacity [21, 22, 83].

Moreover, KV stores do not exploit a cache's biggest advantage: caches are free to evict objects whenever it is convenient. Evicting objects opportunistically can greatly reduce writes and maximize space utilization, but KV stores are not built to exploit this cache-specific optimization.

Existing flash caches do not address DLWA. Because of the unique challenges of flash caching, there is a growing body of work devoted to improving flash cache designs. Prior flash caches generally fall under three categories (Figure 5): log-structured, set-associative, and hierarchical.

Log-structured caches. To minimize writes, many flash caches are log-structured [16, 27, 35, 83]. These caches append objects to an on-flash log (Figure 5(a)), locating objects through a DRAM index and evicting objects in large groups. The log allows large sequential writes to flash and thus achieves nearly ideal write amplification.

While log-structured caches work well for larger objects, the DRAM index becomes prohibitively large for small objects, even if it is highly optimized [67], significantly increasing overall emissions and cost (see Figure 13). Flash caches are thus often partitioned, using a log-structured cache for large objects and a different design for small objects [16].

Set-associative caches. Set-associative caches, such as the Small Object Cache in Meta's CacheLib [16], replace the DRAM index with a hash function that maps each object to a unique set (usually a 4-KB page) on flash (Figure 5(b)).

The downside of these caches is that they cause significantly more writes. When a set-associative cache admits a small object (say, 100 B), it must write at least one flash page (4 KB), resulting in large ALWA (40×). Even worse, these caches perform random writes, leading to DLWA of 2× to 10× [67]. Since write amplification (WA) is the product of ALWA and DLWA, *a set-associative cache's* WA *easily exceeds 100*×. To mitigate this, Meta's flash caches use only 50% of the drive [16], increasing miss ratio and carbon emissions.

Hierarchical. FairyWREN builds on Kangaroo [67, 68], a hierarchical flash cache for small objects that combines a small log-structured cache (KLog) and a large set-associative cache (KSet) (Figure 5(c)). Kangaroo uses KLog to reduce ALWA and KSet to reduce DRAM. Objects are first written to KLog. Once KLog is full, these objects are then rewritten in batches to KSet in a *flush* operation. Since KLog holds many objects, several objects admitted during a flush will map to the

same set in KSet. The ALWA for writing each set is then amortized across all objects that map to that set. In essence, KLog is a buffer that finds set collisions for KSet in order to reduce writes. The larger KLog is, the more collisions will be found during flush operations, lowering KSet's ALWA in exchange for higher DRAM overhead. Kangaroo also employs a selective threshold admission policy to limit which objects are written from KLog to KSet, further reducing flash writes in KSet.

Kangaroo still only needs 5–10% of flash for KLog to substantially reduce KSet's writes. Since KSet comprises more than 90% of the cache capacity, the DRAM needed to index KLog is limited. Kangaroo also uses a highly optimized partitioned index data structure to reduce the DRAM used by KLog. Due to its low DRAM overhead, Kangaroo achieves large emission reductions over a memory-optimized log-structured cache, Flashield [35], for workloads with many small objects (Figure 13 in Section 6.3). This comparison shows that a carbon-efficient cache needs to have a low DRAM overhead.

While Kangaroo greatly reduces writes by limiting ALWA, it still writes too much, because *Kangaroo cannot control device-level write amplification*. Kangaroo experiences high DLWA, because KSet performs random 4-KB writes, the worst case for DLWA on LBAD devices. Because of its high write budget requirements, Kangaroo cannot reduce emissions by moving to denser flash. For example, Figure 4 shows that, for a 10-year lifetime, QLC tolerates only 0.37 **device-writes per day (DWPD)** and PLC tolerates only 0.16 DWPD. Kangaroo performs 1.46 DWPD in our evaluation. Our goal is to build a sustainable cache that achieves Kangaroo's low DRAM usage while also writing far less to flash. We find that flash caches need a different flash interface in order to reduce DLWA without adding DRAM overhead.

4 Write-Read-Erase iNterfaces

Prior flash caches incur excessive DLWA (Section 3). The root causes are the mismatch between write and erase granularities and a legacy LBAD interface that hides this mismatch from software. This section discusses recent WREN, such as ZNS [19] and FDP[66], that include `Erase` as a first-order operation. We show that WREN is necessary but insufficient: A new flash interface does not reduce writes by itself, changes to the cache design are required.

4.1 Today's Interface Is LBAD

Most flash SSDs today are LBAD, sharing the same interface as disks. LBAD presents the flash device as a linear address space of fixed-size blocks[2] that can be independently read or written.

LBAD eased the transition from HDDs to SSDs but does not expose the erase granularity of flash (Section 3). As a result, the LBAD device firmware must perform GC that can cause high DLWA and tail latency. Although there has been work to decrease DLWA [40, 41, 44, 56, 89, 91], LBAD devices still hide erase units and GC from applications, preventing co-optimization to minimize overall flash writes.

4.2 Challenges of New Interface Design

While a variety of flash interfaces have been proposed [20, 44, 51, 52, 72, 78, 88, 96], none have gained widespread adoption. Two proposals, Multi-streamed SSDs and Open-Channel SSDs, illustrate the pitfalls of designing a new flash interface.

Multi-streamed SSDs [51, 52] allow users to direct writes to different *streams*. Streams provide isolation between workloads: Different streams write to different EUs. When objects with similar lifetimes are grouped into the same stream, GC is more efficient. However, because the application does not control GC directly, DLWA remains a significant issue.

[2]These fixed-size blocks correspond to pages, not flash blocks (Section 3).

Open-Channel SSDs [20] remove all flash-device logic and force applications to handle *all* of flash's complexities, even beyond those described in Section 3. While the hope was to develop layers of abstraction in software to hide some of this complexity, this software was never widely deployed.

Lesson for flash caches: An ideal flash interface for caching would allow the cache to control *all* writes, including GC, but still present a simple abstraction to application developers.

4.3 What Makes an Interface WREN?

We call interfaces that delegate `Erase` commands and garbage collection to the host WREN. WREN is defined by three main features:

(1) WREN operations. WREN devices must let applications control which EU their data are placed in and when that EU is erased. Specifically, WREN devices must, at least, have `Write`, `Read`, and `Erase` operations.

These operations can be implemented differently. For example, ZNS [19] and FDP [66] are both WREN. Both interfaces are NVMe standards with strong support from industry and provide an abstraction for writing to an EU.[3] However, they have different philosophies, which can be seen, for instance, in their `Write` operations. ZNS provides either sequential writes to an EU or nameless writes through Zone Append [96]. FDP provides random writes within an EU as long as the application tracks that the number of pages written is less than the EU size. Despite these differences, both provide the control over data placement into EUs required by WREN.

Moreover, the aforementioned Open-Channel interface is also WREN. But Open-Channel SSDs expose the full complexity of the device to the host, which is additional complexity *not* required to reduce a cache's DLWA.

(2) The `Erase` *requirement.* Unlike LBAD, WREN devices do not move live data from an EU before erasing it. Applications are responsible for implementing GC to track and move live data before calling `Erase`. `Erase` is different from a traditional `trim`, because `Erase` targets an entire EU rather than individual pages. Failure to perform correct and timely GC is subject to implementation-specific error handling by the device. A major difference between FDP and ZNS is how they treat violations of `Erase` semantics, but this error behavior is inessential to reducing DLWA and thus beyond WREN.

(3) Multiple, but limited, active EUs. An *active EU* is one that can be written to without being erased. WREN devices support a few active EUs at one time. Since an active EU typically requires a device buffer for the EU's data, the maximum number of active EUs is implementation specific. FairyWREN requires four simultaneously active EUs, which we expect will be supported in the vast majority of WREN devices.

4.4 WREN Alone Is Not a Cure for WA

WREN devices make it easy to perform large, sequential writes with no WA. When writing sequentially, the user can maintain a single active EU and fill the EU completely before activating the next EU. Furthermore, if all writes are large and sequential, then it is generally easy to find an EU consisting of invalid data when GC is required, resulting in low WA.

However, not all caches can perform large, sequential writes. Set-associative flash caches also want low WA but perform small, random writes that incur high DLWA on LBAD devices. One might hope that WREN devices can achieve lower WA. A reasonable first attempt at implementing a

[3]This abstraction is called a *zone* in ZNS and a *reclaim unit* in FDP.

Table 2. Variables in Analytical Model of FIFO+

Variable	Definition
X	Random variable representing number of invalid page in an EU chosen for GC
b	Number of pages in an EU
p	Probability that a page is invalid
k	Number of writes between each GC operation
t	Total number of EUs
u	Number of EUs for user data (does not include overprovisioning)

set-associative cache on WREN is to treat each set as an object in a log-structured store, allowing the cache to write updates sequentially to a single active EU. This naive approach does not reduce WA—it just moves the GC from the device to the cache (see Section 6.6).

The impact of smaller EUs. One idea for mitigating WA under small, random writes is to reduce the EU size, e.g., from a gigabyte to tens of megabytes, by removing error correction between flash blocks. Caches can tolerate removing error correction, because they are not tasked with permanently storing the data; rather, lost bits just translate to misses. Prior systems use smaller EUs to minimize GC [14, 69], because, intuitively, lowering the number of sets per EU creates more EUs that are either mostly invalid (good candidates for GC) or mostly valid (bad candidates for GC that are skipped). However, other prior work that mathematically analyzes the WA of FIFO GC policies[34, 46] has largely ignored the effect of EU size. In fact, this modeling work assumes that changing the EU size will not change the WA from GC. To remedy this discrepancy in prior work, we need to model the WA of a FIFO GC policy for a set-associative cache and capture the effect of EU size.

Modeling of DLWA under Random Writes. Our goal is to model the effect of EU size on DLWA. Specifically, we want to analyze the performance of a FIFO+ GC policy, which selects EUs for garbage collection in FIFO order and skips EUs which contain only valid data. The FIFO+ policy is under a random write workload from the set-associative cache, since the inserted key's hash determines which set to write, a random process assuming a perfect hash function.

We use an approach similar to that in Reference [46] to model the relationship between EU size and DLWA under FIFO+. While several prior papers [34, 46, 87] have noted that DLWA can be approximated using W Lambert functions, this prior work focuses on device overprovisioning rather than on the EU size.

We define X to be the random variable representing the number of invalid pages in an EU that is targeted for garbage collection (as seen in Table 2). Because FIFO+ will erase an EU only if it contains invalid pages, our goal is to approximate $\mathbb{E}[X|X > 0]$. This tells us the number of new pages that can be written every time GC is performed. Hence, if we let b be the number of pages in an EU, then we can compute the DLWA as

$$\text{DLWA} = \frac{b}{\mathbb{E}[X|X > 0]}. \tag{1}$$

Our approximation makes two simplifying assumptions.

First, we assume that each of the b pages in the target EU is invalid independently with probability p. This is reasonable when writes are random and the total number of pages in the device is large. This assumption implies that $X \sim \text{Binomial}(b, p)$. To approximate the expectation of X, we must approximate p.

Second, we assume that an EU is targeted for GC every k writes, where k is a constant. Specifically, we define t to be the total number of EUs in the device and assume $k = t\mathbb{E}[X]$. This is a reasonable approximation, because k is the expected number of writes that occur between GC operations on a given EU and the total number of EUs, t, is large. A particular page will be invalid if at least one of the k writes targets the page. Hence, the probability p that a page is invalid is

$$p = 1 - \left(1 - \frac{1}{ub}\right)^k,$$

where u is the number of *EUs* available to store valid user data. Note that u is typically smaller than t, and $\frac{t}{u}$ represents the amount of overprovisioning in the device.

Combining these assumptions yields

$$\mathbb{E}[X] \approx b \cdot p \approx b\left(1 - \left(1 - \frac{1}{ub}\right)^k\right), \tag{2}$$

$$\approx b\left(1 - \left(1 - \frac{1}{ub}\right)^{t\mathbb{E}[X]}\right). \tag{3}$$

We can rewrite Equation (3) using the W Lambert function to get the following approximation for $\mathbb{E}[X]$:

$$\mathbb{E}[X] = b - \frac{W(bt\left(1 - \frac{1}{ub}\right)^{tb} \ln\left(1 - \frac{1}{ub}\right))}{t \ln\left(1 - \frac{1}{ub}\right)}.$$

To compute $\mathbb{E}[X \mid X > 0]$, we note that

$$\mathbb{E}[X \mid X > 0] = \sum_{i=1}^{b} i \cdot \frac{P(X = i)}{P(X > 0)} = \frac{1}{P(X > 0)} \sum_{i=0}^{b} i \cdot P(X = i)$$

and thus

$$\mathbb{E}[X \mid X > 0] = \frac{\mathbb{E}[X]}{P(X > 0)} = \frac{\mathbb{E}[X]}{1 - (1 - p)^k}.$$

Hence, we now have an approximation that allows us to write DLWA as defined in Equation (1) in terms of the device parameters t, u, and b.

Results of model. We validate our model against simulation in Figure 6, where we run both our simulation and the model with an overprovisioning of 7%. Our approximation (Figure 6) matches simulation results, with a R^2 value of 0.9996.

Our approximation shows that when EU sizes are small, FIFO is more likely to find EUs that are mostly invalid or completely valid. This leads to a lower WA, as expected in prior systems, since these EUs require fewer rewrites of valid data. However, as EUs grow, the WA quickly stabilizes. Thus, the WA does not change for EUs larger than around 256 KB.

Lesson for flash caches: We find that *reducing EU size only improves WA for very small EU sizes.* To realize a significant reduction in WA, the EU size must be tens of KBs, but that is unachievable in current devices (Section 3). Hence, we conclude that WREN alone does not reduce WA for caches. To reduce WA, we must also re-design the cache.

5 FairyWREN Overview and Design

FairyWREN uses WREN to substantially reduce WA by unifying cache admission with garbage collection. The resulting reduction in overall writes lets FairyWREN use denser flash while extending device lifetime to improve sustainability.

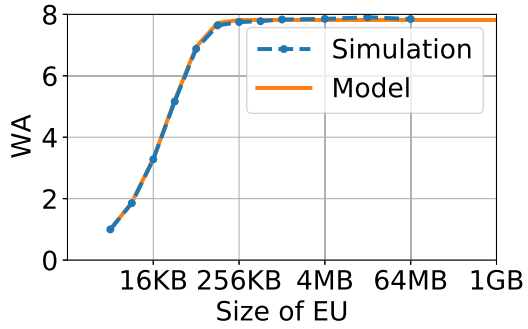


Fig. 6. The DLWA for a set-associative cache running on WREN with 7% overprovisioning. EUs have to be less than 128 KB to significantly reduce DLWA.

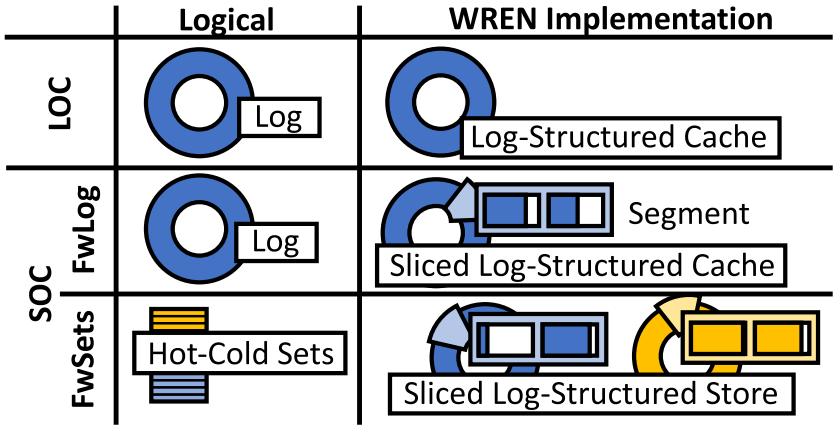


Fig. 7. The components of FairyWREN.

5.1 Overview

How FairyWREN reduces writes. FairyWREN uses WREN's control over data placement and garbage collection to reduce writes in two main ways. First, FairyWREN introduces *nest packing* to combine garbage collection with cache admission and eviction. When live data are rewritten during GC, FairyWREN has an opportunity to evict unpopular objects and admit new objects in their place. In LBAD, by contrast, these objects would have to be rewritten separately for GC and admission/eviction.

Second, FairyWREN groups data with similar lifetimes into the same EU, separating data that in prior caching systems would have been in the same page. If all of the data in each EU have roughly the same lifetime, then EUs will either consist mostly of live data or mostly of dead data. FairyWREN can then GC the mostly dead EUs with few additional writes. FairyWREN leverages two main techniques to enable this grouping: large–small object separation and hot–cold set partitioning.

Architecture of FairyWREN. FairyWREN partitions its capacity into a **large-object cache (LOC)** and a **small-object cache (SOC)**, as seen in Figure 7. Incoming requests first check the LOC and then check the SOC.

The *large-object cache* (Section 5.2) stores objects larger than 2 KB and uses a simple log-structured design, since it can tolerate higher per-object DRAM overhead.

The *small-object cache* (Section 5.3) uses a hierarchical design based on Kangaroo [67]. The SOC contains two levels: FwLog and FwSets. FwLog is a log-structured cache with a relatively high per-object DRAM overhead. The main function of FwLog is to buffer objects so they can be written efficiently to FwSets. Therefore, FwLog can have a fairly low capacity (≈5%), keeping its DRAM overhead low. FwSets is a set-associative cache, but since WREN does not support random writes, the sets are kept in a log-structured store. FwSets stores sets, not individual objects, in the log to minimize DRAM. When this log-structured store is garbage collected, objects are opportunistically moved from FwLog into FwSets. Finally, each set in FwSets is further partitioned into hot (frequently accessed, long-lived) objects and cold (recently admitted, short-lived) objects (Section 5.4).

5.2 The LOC

The LOC is a log-structured cache. Adapting log-structured caches to WREN is straightforward, since they only perform large, sequential writes. The LOC is broken into large segments, each the size of an EU. Segments can then be evicted in LRU or FIFO order with minimal WA. The LOC uses DRAM in two ways: (i) an in-memory, EU-sized buffer for log insertions and (ii) an in-memory index tracking object locations on flash. Because the LOC stores large objects, it contains relatively few objects and needs little DRAM. Besides the segment buffer, all LOC objects are stored on flash.

Insertions. Objects are first inserted into an in-memory segment buffer and added to the in-memory log index. Once the segment buffer is full, it is written to an empty EU in the log.

Lookup. Reads look up the object's key in the log index. If found, then the cache reads the object from the indicated EU.

Eviction. Eventually, the log will fill up and LOC will evict a log segment based on the eviction policy. Since log segments are aligned to EUs, eviction simply `Erase`s an EU, evicting those objects from the cache. This design does not rewrite any objects, incurring minimum WA of 1×.

5.3 The SOC

The focus of FairyWREN is the SOC. Log-structured caches are impractical for caching small objects, because a large flash cache can fit billions of small objects, requiring a large DRAM index to track them all (Section 3.3). FairyWREN's SOC is based on Kangaroo [67], a recent flash cache designed for small objects with low DRAM overhead and low ALWA. The SOC is a hierarchy of two levels: FwLog, a small log-structured cache, and FwSets, a large set-associative cache. FwLog contains about 5% of the SOC's capacity, with the remaining 95% for FwSets. We describe FwLog and FwSets individually, and then how they work together.

FwLog design. FwLog's goal is to buffer new small objects for insertion into FwSets. Like the LOC, FwLog is a log-structured cache that uses an in-memory segment buffer and an in-memory index to track objects in the FwLog. All other objects in the FwLog are stored on flash.

FwSets design. FwSets is a set-associative cache that maps each object to a unique set by hashing its key. When admitting an object, FwSets evicts old objects from the object's set then overwrites it. However, overwriting is impossible in WREN, so FwSets stores *the sets themselves* as objects in a log-structured store. FwSets uses an in-memory index to track the location of each set on flash, but, unlike prior work [56, 61, 78], it does not track individual objects, since this would incur too much DRAM overhead. The index's DRAM overhead is low, because a set is at least 4 KB, whereas objects can be just 10s of bytes. (Larger sets reduce the size of FwSets's DRAM index, but increase average read latency.)

When FwSets's log-structured store is close to full, it must garbage collect in order to admit new objects to the cache. The simplest scheme would be to erase the EU at the tail of the log, evicting all

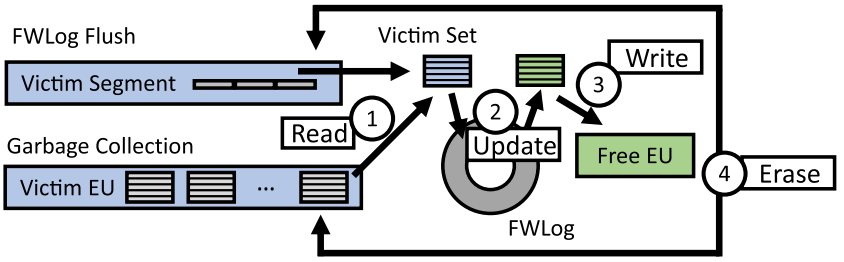


Fig. 8. Nest packing in FairyWREN's small-object cache.

sets—and thus their objects—mapped to this segment.[4] However, since each set contains a mixture of popular and unpopular objects, throwing away entire sets would significantly increase miss ratio. Instead, FwSets rewrites live sets during GC before erasing the EU.

SOC operation. FwLog and FwSets operate as a hierarchy:

Lookup. Lookups first check FwLog for the object. If not found, then FwSets hashes the object's id and looks up the *set's* location. The set is read and scanned for the object.

Insertion. FairyWren first inserts objects into FwLog. When FwLog is full, objects are evicted from FwLog and inserted into FwSets, as described next. Similarly, inserting into FwSets can cause cascading eviction from FwSets.

Eviction (nest packing). If either FwLog or FwSets is running out of space, then FairyWren needs to perform *nest packing* (Figure 8). FairyWren's SOC chooses an EU for eviction from FwLog or FwSets, depending on which is full. If both logs are full, then FwSets is chosen, because FwSets must have space to receive objects evicted from FwLog.

The victim EU is first read into memory. If evicting from FwLog, then each object in the EU hashes to a *victim set*. Otherwise, when evicting from FwSets, each set in the EU *is* a victim set. Then, ① FairyWren rewrites each victim set by the following: ② finding all objects in FwLog that map to a given set, forming a new set containing these objects (evicting objects as necessary), and ③ rewriting the set by appending it to FwSets's log. Finally, ④ FairyWren erases the victim EU.

SOC design rationale. Prior flash caches relied on LBAD GC to reclaim flash space from evicted sets, causing DLWA. *The key difference of FairyWren from prior flash caches is its coordination of cache insertion and eviction with flash GC.*

FairyWren's nest packing algorithm combines previously distinct processes. LBAD caches pay for eviction as ALWA and for garbage collection as DLWA. In the worst case, a set is copied by garbage collection and then is immediately rewritten to admit objects from FwLog. It is impossible to merge these flash writes in LBAD. FairyWren leverages WREN to eliminate unnecessary writes by aligning the eviction and garbage collection cadences of FwLog and FwSets.

5.4 Optimizing the SOC

The SOC is the main source of DRAM overhead and WA in FairyWren. We employ a variety of optimizations to improve the memory and write efficiency of the SOC.

5.4.1 Reducing Flash Writes by Separating Hot and Cold Objects. Even after using nesting to decrease writes, FwSets is still the primary source of flash writes in FairyWren. To further reduce

[4]In this scenario, FwSets would be on a log-structured cache.

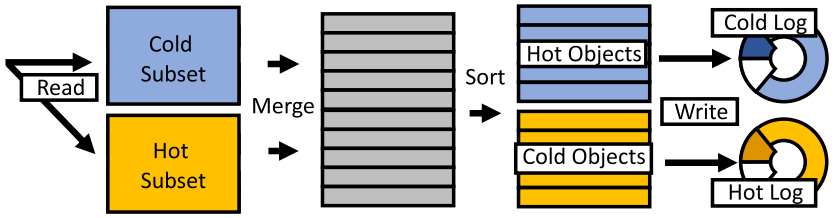


Fig. 9. FwSets is split in two: hot subsets with cold objects and cold subsets with hot objects. Most of the time objects are inserted into the hot subset. However, every n subset updates, both subsets are read, merged, split by object popularity, and then both rewritten.

these writes, FwSets separates objects by popularity, as determined by a modified RRIP algorithm [45, 67]. Instead of a set being *one unit* that is written every insertion, each set in FwSets is split in twain, into a subset for popular objects and a subset for unpopular objects, each backed by its own log-structured store. Each subset is at least a page. Paradoxically, since the unpopular objects are most likely to be evicted, the subsets with unpopular objects correspond to hot (i.e., frequently written) pages on flash. Hence, we refer to the subsets with unpopular objects as *hot subsets* and we refer to the subsets with popular objects as *cold subsets*.

With hot and cold subsets enabled, objects evicted from FwLog are inserted into the hot subset. The cold subset is not typically written during insertion. Every n nest packing operations on a subset, both the hot and cold subsets are read. In memory, these subsets are merged and redivided by object popularity, as seen in Figure 9. Any popular objects found in the hot subset are moved into the cold subset. Since popular objects are likely to remain in the cache for a while, they do not need to be rewritten as frequently. Therefore, they should be in the cold subset and not incur extra rewrites. The least popular objects found in the cold subset are moved into the hot subset so that FwSets can evict them if they remain sufficiently unpopular.

Hot–cold object separation can nearly halve FwSets's write amplification. If n is 5 and sets are 8 KB (two 4-KB subsets), then FairyWren without hot–cold object separation would have to write all 8 KB on each insertion to a set. With hot–cold object separation, FairyWren writes 4 KB for the hot subset on every insertion but only has to write 4 KB for cold subset on every fifth insertion. Specifically, FairyWren writes 4 KB for the first, second, third and fourth new object written to a set, since it only has to update the hot subset with the new object. New objects have a high likelihood of being unpopular, since many objects are never accessed [16] so starting them in the hot subset aligns well with our variant on the RRIP eviction policy. On the fifth insertion, FairyWren remerges the hot and cold subsets—rewriting all 8 KB. Thus, FwSets writes only 24 KB instead of 40 KB every five inserts to a set, a 40% write reduction. Since this write reduction applies to all sets, we see a 40% write reduction for FwSets overall. This translates to a large reduction in FairyWren (Section 6.6).

Theoretically, FairyWren could further reduce writes by further dividing sets. However, there are some practical limitations to this, namely that WREN devices only support a limited number of active EUs, often less than 10. FairyWren currently needs four active EUs: one for LOC, one for FwLog, and two for FwSets (one for the hot subsets and one for the cold subsets). Using only four active EUs allows FairyWren to run concurrently with other programs on the flash without interference and ensures compatibility with a wide range of WREN devices while still achieving low write rates.

Moreover, separating objects by popularity yields diminishing returns, since it increases miss ratio due to object-popularity mispredictions. To maintain miss ratio, the cache then requires

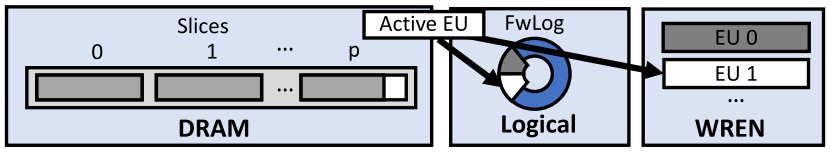


Fig. 10. FwLog uses slicing to minimize memory overhead in FwLog.

more capacity—meaning FairyWren would trade a WA problem, which may require additional capacity to maintain the required write rate, for a just a capacity problem. We expect many wrong object-popularity predictions. FairyWren maintains very few bits of metadata to track each object's popularity to minimize DRAM, leading to low fidelity predictions. The miss ratio will increase if popular objects are placed in hot subsets and evicted prematurely. This type of error becomes more frequent as one tries to separate objects by popularity at finer granularity. In fact, even our single layer of hot–cold separations causes a modest increase in miss ratio (Section 6.6).

5.4.2 Minimizing DRAM in FwLog by Slicing. Like Kangaroo [67, 68], FwLog is implemented as 64 *slices*, i.e., 64 independent log-structured caches that operate in parallel over subsets of the keyspace. This is done to save $\log_2 64 = 6$ bits per flash pointer in the DRAM index.

A naïve implementation of slicing on WREN would require one active EU for each slice. Many WREN devices do not permit 64 simultaneously active EUs due to the prohibitively large DRAM overhead this would impose on the flash device. Instead, FwLog uses a single active EU and shares segments among all 64 slices, giving each slice an equal static share of each segment (Figure 10). The downside of sharing FwLog segments is that one slice could fill up its share of the segment before the others. In the worst case, one slice fills before the others contain any objects, causing internal fragmentation in FwLog. This fragmentation reduces FwLog's ability to minimize WA in FwSets. Via simulation, we found that fragmentation could exceed 20%.

Balls and bins approximation of slicing. To better understand how much fragmentation slicing creates, we model the process of filling a sliced buffer using a balls and bins approximation. Since FwLog hashes each object to a slice, we can model each object as a ball randomly being assigned to a bin representing one slice. To simplify the analysis, we assume each object is the same size.

We want to know how many balls, in expectation, that we can throw before the maximum number of balls in any bin is greater than the number of objects that can fit in a slice (x). To answer this question, we consider the stochastic process of sequentially throwing balls into n bins. It is easy to see that the average number of balls in a given bin is $(\frac{m}{n})$, suggesting that fragmentation should be limited. However, based on our simulation, we know that fragmentation occurs. Thus, we need to bound the deviation of the maximum number of balls in any bin from this mean.

To derive bounds on fragmentation, we define the stochastic process $\{X_m\}$ to be the maximum number of balls in any bin after m balls have been thrown. We define the random variable M to be

$$M = \min_m \{m \mid X_m > x\}.$$

Our goal is to bound $\mathbb{E}[M]$. Fortunately, the results of Raab and Steger [74] give a high-probability bound on X_m which we can use to bound $\mathbb{E}[M]$.

Specifically, Raab and Steger show that $P\{X > k_\alpha\} = o(1)$ if $\alpha > 1$ and $P\{X > k_\alpha\} = 1 - o(1)$ if $0 > \alpha > 1$, when

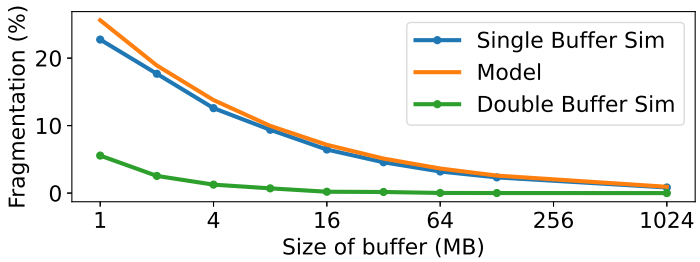


Fig. 11. Comparison of splice model to single and double buffer simulations over a range of buffer sizes with 64 slices ($R^2 = .97$ between single buffer simulation and model).

$$k_\alpha = \begin{cases} \dfrac{\log n}{\log \dfrac{n \log n}{\log m}} \left(1 + \alpha \dfrac{\log\log \dfrac{n \log n}{\log m}}{\log \dfrac{n \log n}{\log m}}\right), & if \dfrac{n}{\text{polylog}(n)} \le m \ll n \log n \\ (d_c - 1 + \alpha) \log n, & \text{if } m = c \cdot n \log n \text{ for some constant } c \\ \dfrac{m}{n} + \alpha \sqrt{2 \dfrac{m}{n} \log n}, & \text{if } n \log n \ll m \le n \,\text{polylog}(n) \\ \dfrac{m}{n} + \sqrt{\dfrac{2m \log n}{n} \left(1 - \dfrac{1}{\alpha} \dfrac{\log\log n}{2 \log n}\right)}, & \text{if } m \gg n(\log n)^3, \end{cases}$$

where $\text{polylog}(x)$ is the class of functions $\bigcup_{i \ge 1} \mathrm{O}\left(\log^i x\right)$ and d_c denotes a suitable constant depending only on c.

In our setup, we only care about the case where $m \gg n(\log n)^3$, since, for 64 slices, $n(\log n)^3 = 377$ and $m \approx 10{,}000$ at least.

To bound $\mathbb{E}[M]$, we note that $P\{X_m > x\} = 1 - o(1)$ if $m \ge k_1$. This gives

$$\begin{aligned} \mathbb{E}[M] &= \mathbb{E}\left[M \mid X_{k_1} > x\right] \cdot P\{X_{k_1} > x\} + \mathbb{E}\left[M \mid X_{k_1} \le x\right] \cdot P\{X_{k_1} \le x\}, && (4) \\ &\ge k_1 \cdot P\{X > k_1\} + 0, && (5) \\ &\ge k_1 \cdot (1 - o(1)). && (6) \end{aligned}$$

Hence, taking limits as n becomes large gives

$$\begin{aligned} \lim_{n \to \infty} \mathbb{E}[M] &\ge k_1 && (7) \\ &= -\frac{\sqrt{n^2(2\log n - \log\log n)(2\log n - \log\log n + 4x)}}{2} - \frac{n \log\log n}{2} + nx + n \log n. && (8) \end{aligned}$$

While Equation (8) is an asymptotic lower bound, we find that it closely matches our simulation results, as seen in Figure 11. Our simulation consists of 100 trials of the balls and bins problem at each buffer size. We plot the average of these 100 trials. We find that, unless the buffer is at least 1 GB, more than 1% of buffered capacity is wasted by our simple buffering policy. Therefore, we need to find another way to decrease our fragmentation without increasing our memory usage.

Leveraging double buffering to decrease fragmentation. FwLog reduces fragmentation via double buffering (Figure 12). On insertion, FwLog ① attempts to insert an object into its slice in the "primary" segment buffer. If the primary is full, then ② the object is inserted into its slice in the secondary, "overflow" segment buffer. ③ When any slice in the overflow buffer becomes more

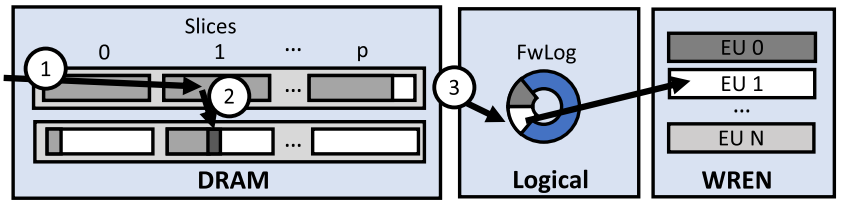


Fig. 12. FwLog uses overflow buffers to ensure the log segments are full when slicing.

than half full, FwLog writes the primary buffer to flash. The overflow buffer then becomes the new primary buffer and vice versa. Double buffering increases the number of objects seen before a buffer is written, reducing the variance in the number of objects in each slice.

Using both simulation, we find that this optimization limits the capacity loss from fragmentation to <1%, even for small (16 MB) buffers (Figure 11). At 16 MB, the double buffer solution has less fragmentation than 1 GB with a single buffer.

5.4.3 Minimizing DRAM in FwSets by Slicing. Like FwLog, FwSets also slices the log-structured store to reduce DRAM overhead, sharing segments to minimize active EUs and segment buffers. However, since sets are much larger than individual objects, the capacity of each bin in our fragmentation model is smaller. This means that FwSets would incur more internal fragmentation than FwLog if using the same buffer size and number of slices. FwSets therefore uses only 8 slices, which keeps fragmentation to less than 1% just like slicing in FwLog.

5.4.4 Reducing DRAM in FwSets by Using Larger Sets. Finally, FwSets further reduces DRAM by using sets larger than 4 KB, reducing the number of sets that need to be tracked proportionally. Naïvely, one might expect that increasing set size would increase flash writes. In a pure set-associative cache, this would be true. However, FwLog buffers objects, and the number of objects that hash to a set also increases proportionally with set size, so FwSets's writes are roughly independent of set size. We see only a 5% increase in WA when going from 8- to 16-KB sets with a 4-KB hot subset and a 12-KB cold subset.

DRAM overhead breakdown. Compared to a LBAD set-associative cache, FwSets requires additional DRAM to track sets. Hot–cold object separation compounds this effect, doubling the number of (sub)sets to track.

Table 3 shows the per-object DRAM overhead for Kangaroo and FairyWREN's SOC. Due to partitioning and double buffering, FairyWREN achieves the same log overhead as Kangaroo. FairyWREN's added overhead shows up in FwSets. Naïvely, when FairyWREN has 4-KB subsets and 200-B objects, each set would need 8 bytes, for 3.1 bits per object. However, since FairyWREN uses 8-KB subsets and slices FwSets in eighths, FwSets needs just 1.4 bits per object to track sets.

FairyWREN uses 19% more DRAM than Kangaroo, a 1.5-GB DRAM overhead increase for a 2-TB cache. However, FairyWREN's DRAM overhead is still much lower than a log-structured cache, and this modest DRAM increase allows FairyWREN to greatly decrease flash writes (by 12.5×), netting large savings in carbon emissions and cost.

6 Evaluation

We compare FairyWREN to prior flash caches and find that (1) FairyWREN reduces flash writes by 92% over the research state-of-the-art Kangaroo, leading to a 33% carbon reduction and a 35% cost reduction; (2) FairyWREN is within 11% of the minimum write rate; and (3) FairyWREN is the first cache design to actually benefit from QLC.

Table 3. Kangaroo and FairyWren's SOC's DRAM Overhead for a 2-TB Small-object Cache with a 5% Log

Component	Kangaroo	Naïve SOC	SOC
Log total	*48 bits/obj*	*48 bits/obj*	*48 bits/obj*
Set index	—	≈ ***3.1 b***	≈ ***1.4 b***
Sets (other)	4 b	4 b	4 b
Sets total	*4 bits/obj*	*7.1 bits/obj*	*5.4 bits/obj*
Log metadata	≈ 0.8 b	≈ 0.8 b	≈ 0.8 b
Log size	5% = 2.4 b	5% = 2.4 b	5% = 2.4 b
Set size	95% = 3.8 b	95% = 6.7 b	95% = 5.1 b
Total	**7.0 bits/obj**	**9.9 bits/obj**	**8.3 bits/obj**

Despite tracking sets, FairyWren's SOC still needs fewer than 10 bits per object.

Table 4. FairyWren and Kangaroo Experiment Parameters

Parameter	FairyWren	Kangaroo
Interface	WREN (ZNS)	LBAD
Flash capacity	400 GB	400 GB
Usable flash capacity	383 GB	376 GB
LOC size	10% of flash	10% of flash
SOC log size	5% of SOC	5% of SOC
SOC set size	4 KB hot, 4 KB cold	4 KB
Hot-set write frequency	every 5 cold set writes	
Set over-provisioning	5%	

6.1 Experimental Setup

Implementation. We implement FairyWren in C++ as a module in CacheLib [16]. All experiments were run on two 16-core Intel Xeon CPU E5-2698 servers running Ubuntu 18.04 with 64 GB of DRAM, using Linux kernel 5.15. For WREN experiments, we use a Western Digital Ultrastar DC ZNS540 1-TB ZNS SSD, using the LOC and ZNS library written by Western Digital [50]. The ZNS SSD has a zone (EU) capacity of 1077 MiB. The devices support 3.5 device writes per day for an expected 5-year lifetime.

We compare to Kangaroo [67] over the first ≈2.5 days of a production trace from Meta. FairyWren uses a ZNS SSD and Kangaroo uses an equivalent LBAD SSD with similar parameters (Table 4). Both caches use 400 GB of flash capacity and achieve similar miss ratios as Kangaroo's production experiments [67]. We overprovision FwSets by 5% to ensure forward progress during nest packing, giving several free EUs to the FwSets log-structured store. Thus, FairyWren effectively uses 383 GB. This idle capacity should decrease in larger flash devices. Kangaroo only uses 376 GB of capacity due to device-level overprovisioning. We approximate Kangaroo's DLWA based on results in the Kangaroo paper [67].

Simulation. In addition to flash experiments, we implemented a simulator to compare a much wider range of possible configurations for FairyWren. The simulator replays a scaled-down trace to measure writes and misses from each level of the cache, including the LOC, FwLog, and FwSets.

We evaluate our cache in simulation on a 21-day trace from Meta [16] and a 7-day trace from Twitter [92]. The Meta trace accesses 6 TB of unique bytes with a 13.8% compulsory miss ratio and

Table 5. Scaling Factors for Different Flash Densities

	SLC	MLC	TLC	QLC	PLC
Write endurance	4.4×	4×	1×	0.32×	0.16×
Capacity discount	3×	1.5×	1×	0.75×	0.6×

We optimistically assume that increasing the bits per cell does not affect emissions or cost.

an average object size of 395 bytes. Small objects (<2 KB) are 95.2% of requests, and these requests account for 60.2% of bytes requested. The Twitter trace accesses 3.5 TB of unique bytes and has a 17.2% compulsory miss ratio and an average object size of 265 bytes. Small objects are >99% of requests, and these requests account for >99% of bytes requested. Both of these traces are higher fidelity than the open source traces [16, 92]. We present results for the last 2 days of the trace.

6.2 Carbon Emissions and Cost Model

We want to evaluate carbon emissions and cost across different caching system. Our model allows different cache configuration, flash densities, and device lifetime. Since we want to compare caching systems, our model assumes that a flash device will have the same caching workload for its entire lifetime and that all flash is purchased at the start of the estimated lifetime.

Our model needs to estimate how much flash each cache needs to account for both the cache's capacity and its writes over the desired lifetime. If the cache capacity cannot accommodate the write rate, then we need to overprovision the flash for the write rate. Thus,

$$\text{Flash Capacity} = \max\left(\text{Cache Capacity}, \frac{\text{Write Rate} * \text{Desired Lifetime}}{\text{Write Endurance}}\right).$$

For example, a 2-TB cache with a 6-year lifetime will require at least 2 TBs of flash, but it may require 2.5 TB of flash to accommodate the cache's write rate over 6 years. LBAD devices use 7% overprovisioning, the standard on datacenter drives [8].

We combine this flash capacity requirement with the cache's DRAM configuration and CPU to estimate both the cost and the carbon emissions, assuming that flash's write endurance is the server's main lifetime constraint. While we believe this constraint is reasonable for shorter lifetimes, other failures will become more common at longer lifetimes (particularly above 10 years). We base our write endurance on Micron 7300 NVMe U.2 TLC SSDs. For other densities, we multiply the TLC write endurance by the write-endurance factors in Table 5, based on Reference [9]. We optimistically assume that different flash densities will have the same cost and emissions per cell; e.g., 1 TB of PLC has the same emissions as 600 GB of TLC (5:3 ratio). Our model can incorporate more data on denser flash if it becomes available.

For cost, we account for both the power and acquisition cost of the flash, DRAM, and CPU. For the flash acquisition cost, we interpolate linearly between the Micron SSD's flash capacities to find a cost for any flash capacity. Cost is normalized to Kangaroo with a 30% miss ratio for the Twitter trace and 20% for Meta.

To determine carbon emissions, we use the ACT model [38] to estimate operational and embodied emissions from CPUs, DDR4 DRAM, and flash.

$$\text{Carbon Emissions} = \text{Operational Emissions} + \text{Embodied Emissions}$$

$$= \sum_{\text{CPU, DRAM, Flash}}^{\text{device}} \left(\text{Energy}_{\text{device}} \times \text{Carbon Intensity} \quad + \quad \frac{(\text{Embodied Emissions})_{\text{device}}}{\text{Desired Lifetime}}\right).$$

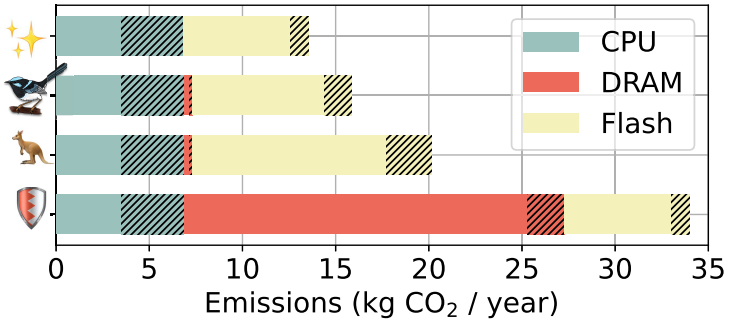


Fig. 13. Yearly carbon emissions for four caching systems: minimum writes (✨) with a write amplification of 1 with no additional DRAM, FairyWren (🐦‍⬛), Kangaroo (🦘), and a Flashield-like log-structured cache (🛡). Our results include the embodied and operational (hatched) emissions from CPU, DRAM, and flash.

For the energy's carbon intensity, we assume the grid is a 50/50 mix of wind and solar, a common renewable-energy mix [12]. The embodied emissions of both DRAM and flash depend on their capacity and we assume that the CPU uses 70% of its maximum power on average.

6.3 Carbon Emissions of Flash Caches

We first examine the carbon emissions of different flash caches for a 6-year deployment. Figure 13 compares FairyWren to three systems: Minimum Writes, Kangaroo, and a Flashield-like log-structured cache [35]. Minimum Writes is an unachievable, idealized cache with WA of 1× and no DRAM overhead. Flashield also assumes a WA of 1× but requires a DRAM:SSD capacity ratio of 1:10, as originally proposed. Since we cannot faithfully replicate Flashield's ML eviction policy (and no working implementation is available), we assume that Flashield achieves FairyWren's miss ratios.

Takeaway 0: Sustainable flash caches must use much less DRAM than log-structured cache designs.

Although we optimistically assumed that Flashield incurs no write amplification, Flashield's overall carbon emissions are 1.7× higher than Kangaroo's. These emissions are due to its high DRAM overhead. Despite optimizations in Flashield designed to save DRAM, high DRAM overhead is unfortunately inherent in the design of a log-structured cache. and thus we need to look beyond log-structured designs.

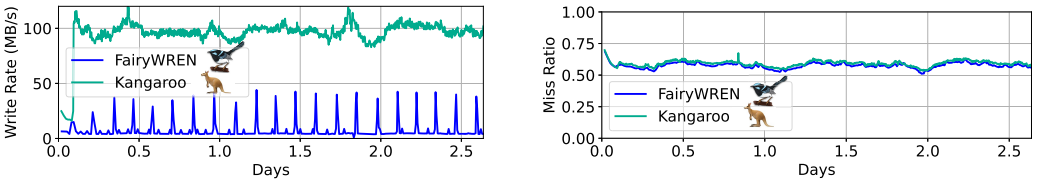
Kangaroo reduces DRAM overhead through its hierarchical design. Unfortunately, Kangaroo also incurs a far higher write rate than a log-structured cache. Kangaroo accounts for its increased writes by overprovisioning flash capacity, increasing the write rate it can maintain in exchange for additional embodied emissions. While Kangaroo is far more sustainable than Flashield, it leaves room for improvement compared to minimum writes due to its overprovisioning needs.

FairyWren maintains Kangaroo's low memory overhead while greatly reducing the flash write rate, lowering its overprovisioning requirements. Consequently, FairyWren reduces overall carbon emissions by 21.2% compared to Kangaroo. As this improvement comes from reducing flash emissions, we focus on flash emissions for the remainder of the evaluation.

6.4 On-flash Experiments

To study how FairyWren reduces flash writes, we evaluate FairyWren on real flash drives using the setup in Section 6.1.

Takeaway 1: FairyWren greatly reduces flash writes while maintaining a slightly better miss ratio than Kangaroo.



(a) Write rate (Mean: FairyWren ≈7.8MB/s, Kangaroo ≈97 MB/s)

(b) Miss ratio (Mean: FairyWren ≈ 0.575, Kangaroo ≈ 0.594)

Fig. 14. The miss ratio and write rate for Kangaroo and FairyWren.

Figure 14 plots the flash write rate and miss ratio over time for Kangaroo and FairyWren. The figure shows small write rate spikes in FairyWren. This is because FairyWren performs nest packing at the granularity of an EU, ≈1 GB. Kangaroo's write rate appears smooth as it flushes more frequently, at 256-KB granularity.

The main goal of FairyWren is to reduce writes, enabling the use of denser flash. In Figure 14(a), FairyWren reduces writes by 12.5× over Kangaroo, from 97 MB/s to 7.8 MB/s. To achieve this, FairyWren leverages WREN to combine cache logic and GC and to separate writes of different lifetimes.

However, reducing writes must not increase misses. Figure 14(b) shows that, in fact, FairyWren and Kangaroo have very similar miss ratios: on average, 0.575 for FairyWren vs. 0.594 for Kangaroo. FairyWren's small advantage comes from reducing idle capacity due to overprovisioning.

We see very similar results for write amplification: a 12.2× reduction, from 23× in Kangaroo to 1.89× in FairyWren. The slight difference between the write rate and WA comes from FairyWren's slightly better miss ratio.

Takeaway 2: FairyWren outperforms Kangaroo for both throughput and read latency at peak load.

While the primary performance metric for caches is miss ratio, FairyWren must provide enough throughput that it does not require more servers—and thus more carbon emissions—to handle the same load. In our experiments, FairyWren's throughput is 104 KOps/s whereas Kangaroo's is 40.5 KOps/s. FairyWren's significant throughput increase is mostly due to lower write rate, but also due to better engineering that moved work off the critical path for lookups and inserts.

Similarly, we find that FairyWren's and Kangaroo's 99th-percentile latencies are 170 µs and 1,370 µs, respectively. But note that, in practice, the overall tail latency is set by the backing store, not the flash cache.

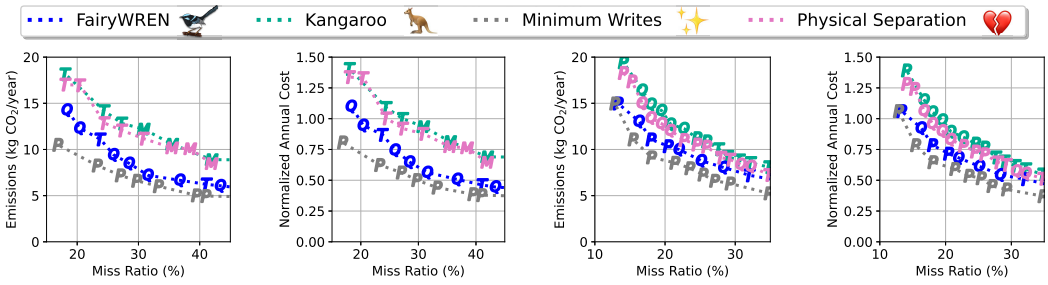
6.5 FairyWren Reduces Carbon Emissions

We now evaluate flash carbon emissions and cost via simulation, comparing FairyWren (🐦‍⬛), Kangaroo (🦘), Minimum Writes (✨), and Physical Separation (💔). Physical Separation represents Kangaroo on WREN, where each cache component (e.g., LOC, KLog, and KSet) is placed in its own EU to separate traffic and thereby allow LOC and KLog to have WA of 1×.

Takeaway 3: FairyWren's reduced writes translate into reduced carbon emissions and reduced cost across miss ratios.

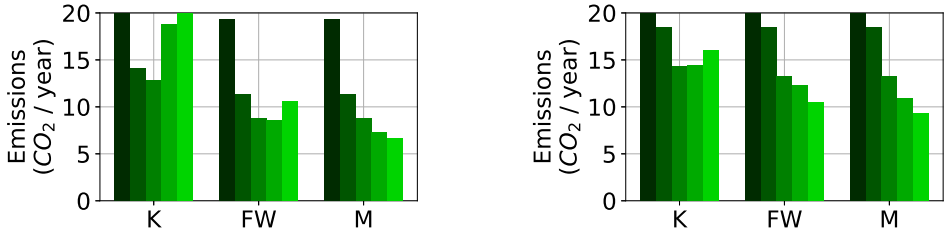
Figure 15 plots emissions and cost for a 6-year lifetime vs. miss ratio over a wide range of cache configurations. Each point is labeled with the flash density used (e.g., T for TLC).

For the Twitter traces (Figure 15(a)–(b)), Kangaroo is limited to either MLC or TLC due to its high write rate, and likewise for Physical Separation, because it does not reduce writes by much (Section 6.6). Meanwhile, FairyWren leverages its low WA to use mostly QLC across miss ratios,



(a) Twitter Emissions *(b) Twitter Cost* *(c) Meta Emissions* *(d) Meta Cost*

Fig. 15. The emissions and cost over 6 years for Kangaroo ( ), FairyWren ( ), Min. Writes ( ), and Physical Sep. ( ).



(a) Twitter *(b) Meta*

Fig. 16. The carbon emissions to achieve a 30% miss ratio on Twitter trace or 20% miss ratio on Meta trace on different flash densities for a desired lifetime of 6 years. Each bar for each cache represents a different density from SLC (left, darkest) to PLC (right, lightest).

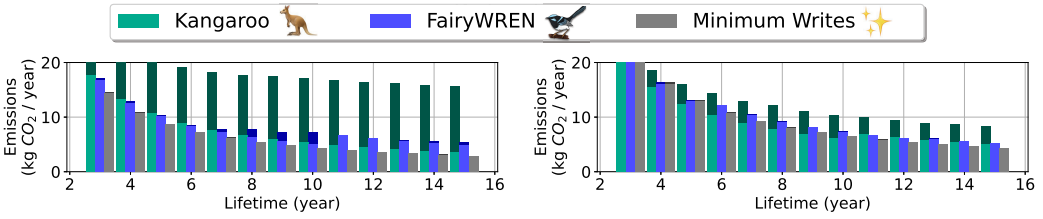
giving it large carbon and cost reductions vs. Kangaroo. However, FairyWren still has too many writes to use PLC. While the gap between Minimum Writes and FairyWren grows at low miss ratios, there is only a 10.1% difference in their emissions at 20% miss ratio and a 7.7% difference in cost.

The Meta traces (Figure 15(c)–(d)) are less write intensive. However, even here we see that FairyWren reduces cache emissions and cost compared to both Kangaroo and Physical Separation. In this case, FairyWren is able to lower the write rate sufficiently to use QLC and PLC. As a result, FairyWren performs close to Minimum Writes, even at low miss ratios.

Takeaway 4: FairyWren benefits from using denser flash when Kangaroo cannot.

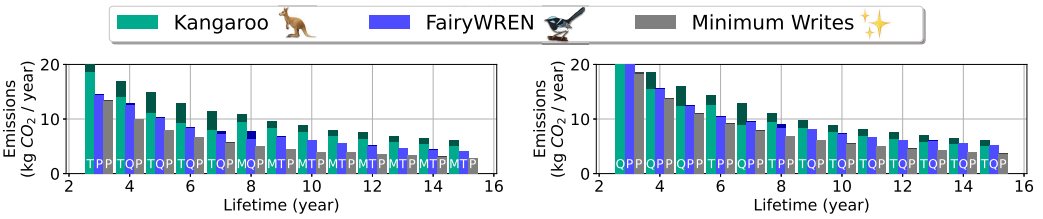
Flash devices are becoming denser over time (Section 2). Figure 16 shows the carbon-optimal cache configurations over a 6-year lifetime at a target miss ratio of 30% for Twitter and 20% for Meta, varying flash density from SLC (left) to PLC (right). Kangaroo performs best when using TLC on the Twitter trace and QLC on the Meta trace. Using PLC increases Kangaroo's emissions due to the excessive overprovisioning needed to compensate for PLC's lower write endurance. FairyWren's lower write rate enables it to use QLC for Twitter and PLC for Meta, reducing emissions and cost. Since Twitter's trace is more write intensive, using PLC increases carbon emissions by 24% due to overprovisioning.

For Minimum Writes on Twitter, emissions decrease by 17% going from TLC to QLC and by 8% from QLC to PLC. On Meta, emissions reduce by 18% and 15%. While these numbers show that denser flash reduces emissions, they suggest diminishing returns even for an optimal cache.



(a) Twitter *(b) Meta*

Fig. 17. The carbon emissions to achieve a 30% miss ratio on Twitter trace or 20% miss ratio on Meta trace with different lifetimes on QLC flash. The darker part of each bar represents emissions due to overprovisioning.



(a) Twitter *(b) Meta*

Fig. 18. The lowest carbon emissions to achieve a 30% miss ratio on Twitter trace or 20% miss ratio on Meta trace while varying both desired lifetimes and flash density. The darker part of each bar represents emissions due to overprovisioning. Letters on each bar represent the flash density.

Takeaway 5: FairyWREN's low WA allows it to avoid massive overprovisioning on dense flash as lifetime is increased.

To explore the trend of increasing device lifetime (Section 2), Figure 17 considers the emissions for caches on QLC devices, showing emissions from overprovisioning in a darker shade.

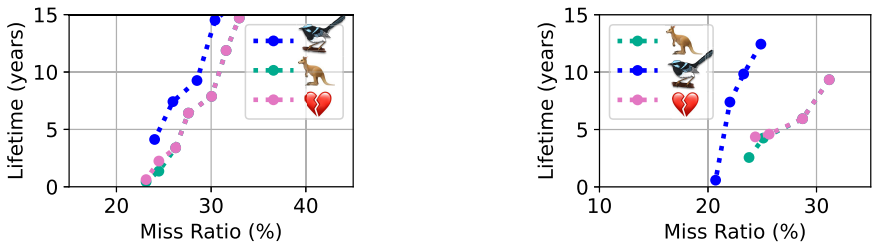
For a 6-year lifetime, Kangaroo requires 2.2× the emissions of FairyWREN on Twitter and 1.17× on Meta. At 12 years, the gap increases to 2.6× and 1.54×. Due to the DLWA in LBAD devices, Kangaroo's emissions are lowest when it has some amount of overprovisioning. FairyWREN does not need this overprovisioning due to its lower WA. This lower overprovisioning leads to FairyWREN's much lower emissions, particularly for the Twitter trace.

Takeaway 6: Increasing flash density does not necessarily improve sustainability, as lifetime matters more than density.

To minimize emissions, we need to optimize both lifetime and flash density. Figure 18 shows each system's emissions for all lifetimes, with the best density displayed on each bar. Kangaroo usually prefers MLC and TLC, because, to provide enough write endurance. QLC and PLC require too much overprovisioning, and thus Kangaroo would have higher emissions if using them. FairyWREN has fewer emissions than Kangaroo at all lifetimes and stays within 30% of Minimum Writes.

The best flash density decreases for longer lifetimes. FairyWREN prefers PLC on Twitter for a 3-year desired lifetime, but TLC for 9 years. At these long lifetimes, the reduced write endurance of denser flash outweighs its sustainability benefits, and extending lifetime is more important than using denser flash. Although a minimum write cache can use PLC for up to 15 years, even a slighly higher write rate quickly overcomes PLC's limited write endurance.

Takeaway 7: For a given flash device, FairyWREN extends lifetime by at least a couple of years.



(a) Twitter *(b) Meta*

Fig. 19. The lifetimes for a 3.6 TB cache for Kangaroo ( ), FairyWren ( ), and Physical Separation ( ).

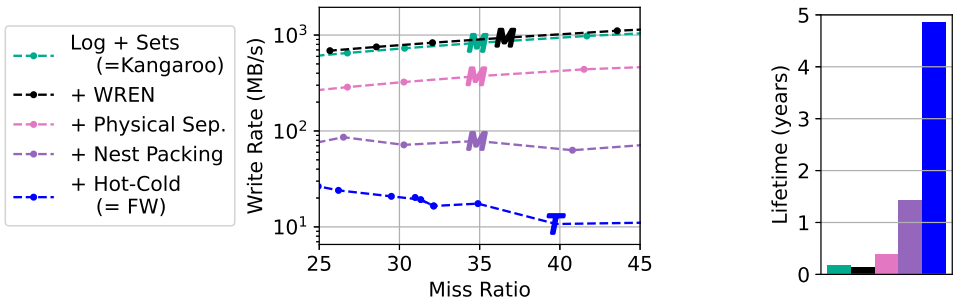


Fig. 20. Write rate (log-scale) and lifetime breakdown on the Twitter trace, incrementally adding optimizations to go from Kangaroo to FairyWren.

So far, we have evaluated emissions when deploying the optimal drive for a given lifetime and flash density. However, flash deployments are often constrained to specific devices with a pre-determined capacity and density. In these situations, extending lifetime can still reduce emissions. Figure 19 evaluates device lifetime for a 3.6-TB drive at different miss ratios. Compared to Kangaroo, FairyWren is able to extend the device's lifetime by at least 2 years and by over 5 years on the Meta trace. By contrast, Physical Separation barely improves lifetime vs. Kangaroo. While Physical Separation reduces writes some over Kangaroo, both ultimately need to massively overprovision to extend lifetime—thus, increasing their miss ratio for any lifetime.

6.6 Where Are Benefits Coming from?

We next explore how FairyWren's optimizations contribute to its write rate reduction. Figure 20 shows the write rate on the Twitter trace starting with Kangaroo on LBAD (Log + Sets). We then add the optimizations of FairyWren incrementally. First, we port Kangaroo naively to WREN (+WREN), then we physically separate the large and small objects into different erase units (+Physical Sep.). Then we add nest packing (+Nest Packing) and, finally, hot–cold object separation (+Hot-Cold) to realize FairyWren. We first present the write rates for the different systems across different capacities and miss ratios, showing the emissions-optimal flash density for one capacity. We then show how the lifetimes of each design would vary if deployed on a QLC drive.

Takeaway 8: Caches on optimal LBAD devices cannot achieve the same write rate as FairyWren.

Three of the lines in Figure 20 are achievable with LBAD devices: Log + Sets, +WREN, and +Physical Sep (though +Physical Sep assumes an augmentation to LBAD such as streams). Log +

Sets represents the current Kangroo implementation on LBAD. +WREN is a naive port of Kangaroo to WREN devices that redirects all cache writes to a single log-structured store using FIFO garbage collection. This naïve port does not attempt any separation of objects by expected lifetime, and we assume it has the same ALWA as Kangaroo. +WREN has a simplistic FIFO garbage collection policy, meaning that it can be worse than just running on LBAD which often do try to separate objects belonging to different streams. This means +WREN has higher write rates than Kangaroo on LBAD. In practice, even the best LBAD implementation must perform somewhere between +WREN and +Physical Sep, which would require LBAD to perfectly predict different streams of data. But even in this best case of Physical Sep., the cache still incurs far too many writes, limiting the lifetime of a QLC device to less than half a year.

Takeaway 9: Both nest packing and hot–cold object separation are essential to FairyWREN's write reduction.

The other two systems we compare in this breakdown are +Nest packing and +Hot-Cold (i.e., FairyWREN with all optimizations). Nest packing reduces writes by at least 3.7× and hot–cold object separation reduces writes by another 3.4×. Either of these optimizations alone would not achieve a close to 5 year lifetime, meaning that the cache still has too many writes to achieve a reasonable deployment lifetime today on QLC. With both optimizations, FairyWREN achieves up to a 33× increase in QLC lifetime over the Kangaroo baseline and a 13× increase over +Physical Sep. We also observe that, even though hot–cold separation can increase miss ratios, the reduction in write rate and its accompanying reduction in overprovisioning outweighs this miss ratio increase.

6.7 Operating on a Fixed Flash Device

We now compare Kangaroo and FairyWREN with respect to miss ratio given a fixed flash capacity. We enforce the same constraints of a 6-year flash lifetime, TLC flash density, and 32 GB of DRAM for both systems. Unlike prior figures where we minimize emissions, FairyWREN cannot not gain an advantage for using denser flash, and Kangaroo cannot increase write endurance by using less-dense flash. We show that FairyWREN under the same capacity constraints, and thus write rate constraints, improves miss ratio over Kangaroo through its reduction in writes allowing it to more effectively use the capacity.

Takeaway 10: FairyWREN achieves the same miss ratio at lower flash capacities than Kangaroo.

Figure 21 shows the effects of changing the flash capacity on miss ratio for both traces. For each flash capacity, we also plot the write rate and WA of both systems. We find that FairyWREN needs less flash capacity than Kangaroo to achieve a given miss ratio. FairyWREN also requires less overprovisioning due to its lower write rate. This trend is more prominent in the Twitter trace than the Meta trace, which is less write intensive. For the Twitter trace, the limitation of only using TLC prevents Kangaroo from achieving better miss ratios, since Kangaroo's needs much more overprovisioning, increasing the overall flash capacity needed to survive 6 years above 3.6 TB. Thus, Kangaroo's miss ratio curve shifts to the right.

We also see that flash capacity sets the write budget for the flash device, defining the write rate that the caching system can tolerate for a desired lifetime. As the capacity increases, both FairyWREN and Kagnaroo can maintain a higher write rate and both systems use that write rate to further reduce misses. One might expect a similar relationship for write amplification. However, the systems have different miss ratios, causing Kangaroo to need to have a lower WA through massive overprovisioning.

Takeaway 11: FairyWREN maintains its advantage under a DRAM constraint.

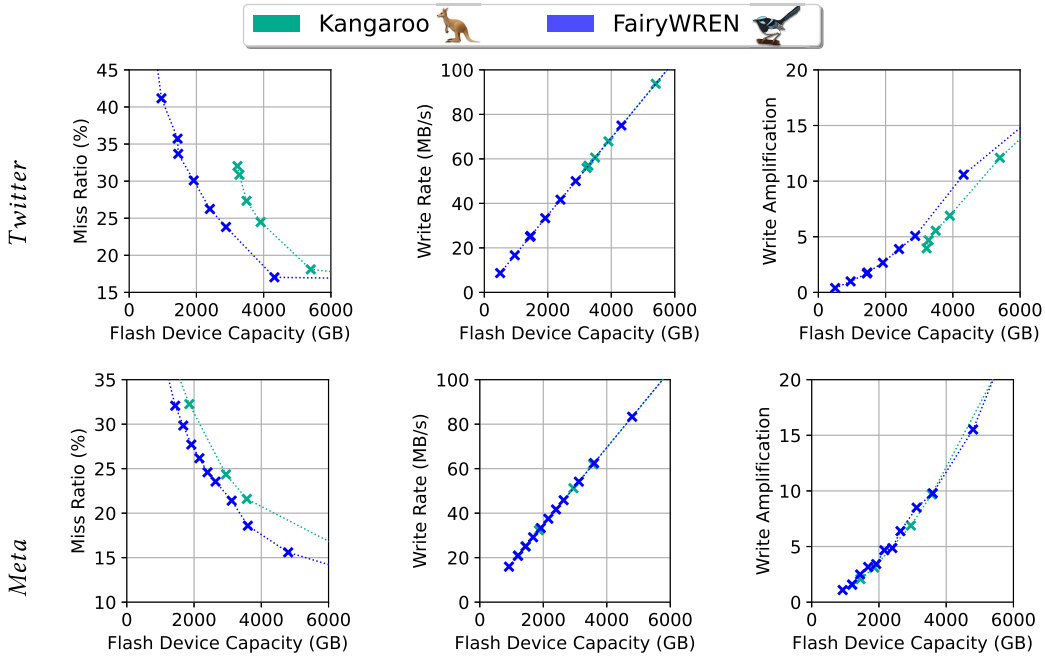


Fig. 21. Pareto curve of cache miss ratio at different flash device sizes and the corresponding write rate and write amplification of these points. The DRAM capacity is limited to 32 GB, the desired lifetime is 6 years, and the caches use TLC flash.

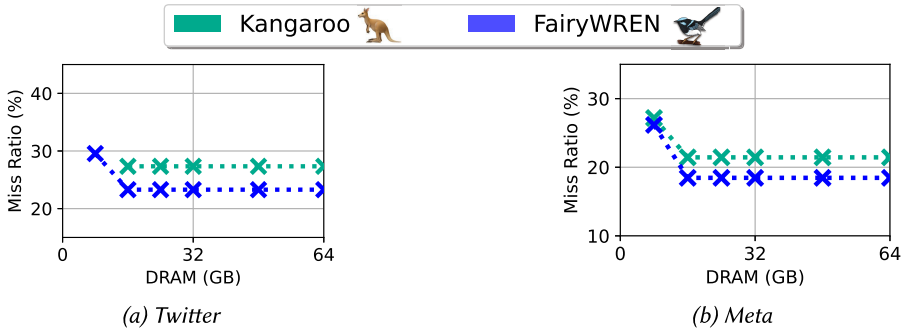


Fig. 22. Pareto curve of cache miss ratio at different DRAM sizes. The flash capacity is limited to 3.6 TB, the desired lifetime is 6 years, and the caches use TLC flash.

We investigated how DRAM restrictions affect Kangaroo and FairyWren when both caches use 3.6 TB of TLC flash for a 6-year lifetime, Figure 22. Despite having a large DRAM footprint, FairyWren maintains a constant miss ratio advantage over Kangaroo from 16 to 64 GB of DRAM for both traces. FairyWren still has a low-enough overhead to need less than 16 GB of DRAM for a full 3.6 TB on-flash cache. Therefore, similarly to less DRAM-constrained environments, FairyWren's lower write rate translates directly into using more cache capacity and a lower miss ratio.

FairyWren's miss ratio only begins to increase when DRAM falls to 8 GB on the both traces. On the Twitter workload, Kangaroo cannot handle the workload with only 8 GB of DRAM. As seen in the Figure 21, with too small of a cache, Kangaroo actually needs more overprovisioning to handle the extra writes from the higher miss ratio. With the DRAM overhead too high to enable a larger cache, Kangaroo cannot be configured to run with 8 GB of memory and only 3.6 TB of flash capacity. Even for the Meta workload with its lower writes, we see that FairyWren performs slightly better than Kangaroo at 8 GB of DRAM. FairyWren's slightly higher DRAM overhead means its cache capacity is more constrained than Kangaroo's, but its lower overprovisioning results in a slightly lower miss ratio. Hence, FairyWren always outperforms Kangaroo even under severe memory constraints.

7 Related Work

This section discusses additional related work with similar techniques and goals to FairyWren.

Hot–cold objects and deathtime. In caching, hot objects are the most popular objects. Caches use eviction policies to retain popular objects [15, 45, 47, 83]. FairyWren adapts Kangaroo's RRIP-based eviction policy [45, 67].

Popularity is different than *deathtime*, the time when an object will be deleted [41]. To minimize GC, many storage systems will physically separate objects by their deathtime [26, 28, 41, 54, 76, 94]. Grouping objects with similar deathtimes reduces WA. Hence, accurately predicting deathtimes is vital for minimizing write amplification within LBAD. Recent work uses ML to make these predictions [26, 94]. Unfortunately, ML solutions require additional hardware that can increase emissions and cost.

Caches have more control over deathtimes than storage systems. Deathtimes are set by the eviction policy, and thus determining an object's deathtime is more straightforward. For instance, in caches that evict based on TTLs, the TTLs can be used to group objects [93]. FairyWren leverages its eviction policy's popularity rankings and the WREN interface to physically group objects by deathtime.

Eviction and garbage collection. Prior flash caches have attempted to reduce in-device garbage collection. Many log-structured caches [27, 35, 56, 61] group objects into large segments and trim these segments during eviction to minimize garbage collection. These systems attempt to evict segments before device-level GC rewrites them. Unfortunately, this does not ensure GC is prevented on LBAD devices, so some work has proposed leveraging newer interfaces to guarantee alignment. DidaCache [78], for example, uses an Open-Channel SSD [20] to guarantee its segments will align with erase units. Other proposals to use more expressive interfaces re-implement LBAD-like GC on top of a ZNS SSD [29], prohibiting optimizations like FairyWren's nest packing. All of these log-structured approaches suffer from high DRAM overheads and cannot evict individual objects without additional writes.

Grouping by object size. FairyWren separates objects into two object size classes, large and small, similar to Kangaroo [68] and CacheLib [16]. This grouping is used to minimize memory overhead. Allocating memory using size-based slab classes is often used to reduce fragmentation [25, 43, 77, 78, 93]. Introducing additional object size classes in FairyWren would result in additional flash accesses, since FairyWren does not index the size classes to save memory. Instead, FairyWren reduces fragmentation by grouping objects into either large segments in the LOC or sets in FwSets. These segments and sets are periodically rearranged to prevent fragmentation.

8 Conclusion

FairyWren reduces flash's carbon emissions and cost by integrating flash management with cache policies. Doing so requires redesigning the cache to transition from old LBAD flash interfaces to a WREN interface. Experiments show that FairyWren decreases flash writes by 12.5× vs. the state of the art, allowing longer flash lifetimes that reduce carbon emissions by 33% and cost by 35%.

Acknowledgements

We thank the members and companies of the PDL Consortium (Amazon, Datadog, Google, Hitachi, Honda, IBM Research, Intel, Jane Street, Meta, Microsoft Research, Oracle, Pure Storage, Salesforce, Samsung, Two Sigma, Western Digital) for their interest, insights, feedback, and support. We thank Gala Yadgar and our anonymous reviewers for their helpful comments and suggestions. We also thank Western Digital for providing resources and technical expertise; we especially thank Matias Bjørling, Ajay Joshi, and Hans Holmberg. We also specifically thank Javier Gonzalez and Mike Allison, at Samsung, and Ross Stenfort, at Meta, for providing their technical expertise on FDP. We thank the PDL staff, particularly Jason Bowles, for their support.

References

[1] Amazon Sustainability. n.d. Retrieved from https://sustainability.aboutamazon.com/climate-solutions
[2] Climate Change Is Humanity's Mext Big Moonshot. n.d. Retrieved from https://blog.google/outreach-initiatives/sustainability/dear-earth/
[3] Fatcache. n.d. Retrieved from https://github.com/twitter/fatcache
[4] Flash Prices. n.d. Retrieved from https://jcmit.net/flashprice.htm
[5] LevelDB. n.d. Retrieved from https://github.com/google/leveldb
[6] Memory Prices. n.d. Retrieved from https://jcmit.net/memoryprice.htm
[7] RocksDB. n.d. Retrieved from http://rocksdb.org
[8] SSD Over-Provisioning and Its Benefits. n.d. Retrieved from https://www.seagate.com/blog/ssd-over-provisioning-benefits-master-ti/
[9] WD and Tosh Talk Up Penta-level Cell Flash. n.d. Retrieved from https://blocksandfiles.com/2019/08/07/penta-level-cell-flash/. 5/17/22.
[10] 2023. Is There a Limited Warranty for Samsung SSDs? Retrieved from https://semiconductor.samsung.com/us/consumer-storage/support/faqs/05/
[11] 2023. Our Path to Net Zero. Retrieved from https://sustainability.fb.com/wp-content/uploads/2023/07/Meta-2023-Path-to-Net-Zero.pdf
[12] Bilge Acun, Benjamin Lee, Fiodar Kazhamiaka, Kiwan Maeng, Udit Gupta, Manoj Chakkaravarthy, David Brooks, and Carole-Jean Wu. 2023. Carbon explorer: A holistic framework for designing carbon aware datacenters. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. ACM, New York, NY, 118–132. https://doi.org/10.1145/3575693.3575754
[13] Nitin Agrawal, Vijayan Prabhakaran, Ted Wobber, John D. Davis, Mark Manasse, and Rina Panigrahy. 2008. Design tradeoffs for SSD performance. In *Proceedings of the USENIX Annual Technical Conference (ATC'08)*. USENIX Association, Berkeley, CA, 57–70.
[14] Hanyeoreum Bae, Jiseon Kim, Miryeong Kwon, and Myoungsoo Jung. 2022. What you can't forget: Exploiting parallelism for zoned namespaces. In *Proceedings of the 14th ACM Workshop on Hot Topics in Storage and File Systems (HotStorage'22)*. Association for Computing Machinery, New York, NY, 79–85. https://doi.org/10.1145/3538643.3539744
[15] Nathan Beckmann, Haoxian Chen, and Asaf Cidon. 2018. LHD: Improving hit rate by maximizing hit density. In *Proceedings of the USENIX Symposium on Networked Systems Design and Implementation (NSDI'18)*.
[16] Benjamin Berg, Daniel S. Berger, Sara McAllister, Isaac Grosof, Sathya Gunasekar, Jimmy Lu, Michael Uhlar, Jim Carrig, Nathan Beckmann, Mor Harchol-Balter, and Gregory G. Ganger. 2020. The CacheLib caching engine: Design and experiences at scale. In *Proceedings of the USENIX Symposium on Operating Systems Design and Implementation (OSDI'20)*.
[17] Daniel S. Berger, Fiodar Kazhamiaka, Esha Choukse, Inigo Goiri, Celine Irvene, Pulkit A. Misra, Alok Kumbhare, Rodrigo Fonseca, and Ricardo Bianchini. 2023. Research avenues towards net-zero cloud platforms. In *Proceedings of the Workshop on NetZero Carbon Computing* (2023).

[18] Daniel S. Berger, Ramesh K. Sitaraman, and Mor Harchol-Balter. 2017. AdaptSize: Orchestrating the hot object memory cache in a content delivery network. In *USENIX Symposium on Networked Systems Design and Implementation (NSDI'17)*.

[19] Matias Bjørling, Abutalib Aghayev, Hans Holmberg, Aravind Ramesh, Damien Le Moal, Gregory R. Ganger, and George Amvrosiadis. 2021. ZNS: Avoiding the block interface tax for flash-based SSDs. In *Proceedings of the USENIX Annual Technical Conference (USENIX ATC'21)*. USENIX Association, 689–703.

[20] Matias Bjørling, Javier Gonzalez, and Philippe Bonnet. 2017. Lightnvm: The linux open-channel SSD subsystem. In *USENIX Conference on File and Storage Technologies*. USENIX, Berkeley, CA, 359–374.

[21] Netflix Technology Blog. 2016. Application Data Caching using SSDs. Retrieved from https://netflixtechblog.com/application-data-caching-using-ssds-5bf25df851ef. (2016).

[22] Netflix Technology Blog. 2018. Evolution of Application Data Caching: From RAM to SSD. Retrieved from https://bit.ly/3rN73CI. (2018).

[23] Simona Boboila and Peter Desnoyers. 2010. Write endurance in flash drives: Measurements and analysis. In *Proceedings of the USENIX Conference on File and Storage Technologies (FAST'10)*.

[24] Erik Brunvand, Donald Kline, and Alex K. Jones. 2018. Dark silicon considered harmful: A case for truly green computing. In *Proceedings of the 9th International Green and Sustainable Computing Conference (IGSC'18)*.

[25] Daniel Byrne, Nilufer Onder, and Zhenlin Wang. 2019. Faster slab reassignment in memcached. In *Proceedings of the International Symposium on Memory Systems (MEMSYS'19)*. Association for Computing Machinery, New York, NY, 353–362. https://doi.org/10.1145/3357526.3357562

[26] Chandranil Chakraborttii and Heiner Litz. 2021. Reducing write amplification in flash by death-time prediction of logical block addresses. In *Proceedings of the 14th ACM International Conference on Systems and Storage*. ACM, New York, NY, 1–12. https://doi.org/10.1145/3456727.3463784

[27] Badrish Chandramouli, Guna Prasaad, Donald Kossmann, Justin Levandoski, James Hunter, and Mike Barnett. 2018. FASTER: An embedded concurrent key-value store for state management. *Proc. VLDB* 11, 12 (2018). 1930.

[28] Mei-Ling Chiang, Paul C. H. Lee, and Ruei-Chuan Chang. 1999. Using data clustering to improve cleaning performance for flash memory. *Softw.: Pract. Exp.* 29, 3 (1999), 267–290.

[29] Gunhee Choi, Kwanghee Lee, Myunghoon Oh, Jongmoo Choi, Jhuyeong Jhin, and Yongseok Oh. 2020. A new LSM-style garbage collection scheme for ZNS SSDs. In *Proceedings of the 12th USENIX Workshop on Hot Topics in Storage and File Systems (HotStorage'20)*. USENIX Association.

[30] Amanda Peterson Corio. n.d. Five Years of 100% Renewable Energy—And a Look Ahead to a 24/7 Carbon-free Future. Retrieved from https://cloud.google.com/blog/topics/sustainability/5-years-of-100-percent-renewable-energy

[31] Niv Dayan, Manos Athanassoulis, and Stratos Idreos. 2017. Monkey: Optimal navigable key-value store. In *Proceedings of the ACM International Conference on Management of Data (SIGMOD'17)*. https://doi.org/10.1145/3035918.3064054

[32] Niv Dayan and Stratos Idreos. 2018. Dostoevsky: Better space-time trade-offs for LSM-tree based key-value stores via adaptive removal of superfluous merging. In *Proceedings of the ACM International Conference on Management of Data (SIGMOD'18)*. https://doi.org/10.1145/3183713.3196927

[33] Biplob Debnath, Sudipta Sengupta, and Jin Li. 2011. SkimpyStash: RAM space skimpy key-value store on flash-based storage. In *Proceedings of the ACM International Conference on Management of Data (SIGMOD'11)*.

[34] Peter Desnoyers. 2012. Analytic modeling of SSD write performance. In *Proceedings of the 5th Annual International Systems and Storage Conference*. 1–10.

[35] Assaf Eisenman, Asaf Cidon, Evgenya Pergament, Or Haimovich, Ryan Stutsman, Mohammad Alizadeh, and Sachin Katti. 2019. Flashield: A hybrid key-value cache that controls flash write amplification. In *Proceedings of the USENIX Symposium on Networked Systems Design and Implementation (NSDI'19)*.

[36] Alex Gartrell, Mohan Srinivasan, Bryan Alger, and Kumar Sundararajan. McDipper: A Key-value Cache for Flash Storage. n.d. Retrieved from https://www.facebook.com/notes/facebook-engineering/mcdipper-a-key-value-cache-for-flash-storage/10151347090423920/

[37] Saugata Ghose, Abdullah Giray Yaglikçi, Raghav Gupta, Donghyuk Lee, Kais Kudrolli, William X. Liu, Hasan Hassan, Kevin K. Chang, Niladrish Chatterjee, Aditya Agrawal, et al. 2018. What your DRAM power models are not telling you: Lessons from a detailed experimental study. *Proc. ACM Meas. Anal. Comput. Syst.* 2, 3, Article 38 (Dec. 2018), 41 pages. https://doi.org/10.1145/3224419

[38] Udit Gupta, Mariam Elgamal, Gage Hills, Gu-Yeon Wei, Hsien-Hsin S. Lee, David Brooks, and Carole-Jean Wu. 2022. ACT: Designing sustainable computer systems with an architectural carbon modeling tool. In *Proceedings of the 49th Annual International Symposium on Computer Architecture*. ACM.

[39] Udit Gupta, Young Geun Kim, Sylvia Lee, Jordan Tse, Hsien-Hsin S. Lee, Gu-Yeon Wei, David Brooks, and Carole-Jean Wu. 2021. Chasing carbon: The elusive environmental footprint of computing. In *Proceedings of the IEEE International Symposium on High-Performance Computer Architecture (HPCA'21)*. IEEE, 854–867.

[40] Mingzhe Hao, Levent Toksoz, Nanqinqin Li, Edward Edberg Halim, Henry Hoffmann, and Haryadi S. Gunawi. 2020. LinnOS: Predictability on unpredictable flash storage with a light neural network. In *Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI'20)*. USENIX Association, 173–190. https://www.usenix.org/conference/osdi20/presentation/hao

[41] Jun He, Sudarsun Kannan, Andrea C. Arpaci-Dusseau, and Remzi H Arpaci-Dusseau. 2017. The unwritten contract of solid state drives. In *Proceedings of the 15th European Conference on Computer Systems (ACM EuroSys'17)*.

[42] Amy Hood. July 2022. (July 2022).

[43] Xiameng Hu, Xiaolin Wang, Yechen Li, Lan Zhou, Yingwei Luo, Chen Ding, Song Jiang, and Zhenlin Wang. 2015. LAMA: Optimized locality-aware memory allocation for key-value cache. In *Proceedings of the USENIX Annual Technical Conference (USENIX ATC'15)*. USENIX Association, Berkeley, CA, 57–69. https://www.usenix.org/conference/atc15/technical-session/presentation/hu

[44] Jian Huang, Anirudh Badam, Laura Caulfield, Suman Nath, Sudipta Sengupta, Bikash Sharma, and Moinuddin K. Qureshi. 2017. FlashBlox: Achieving both performance isolation and uniform lifetime for virtualized SSDs. In *Proceedings of the 15th USENIX Conference on File and Storage Technologies (FAST'17)*. USENIX Association, Berkeley, CA, 375–390. https://www.usenix.org/conference/fast17/technical-sessions/presentation/huang

[45] Aamer Jaleel, Kevin Theobald, Simon Steely Jr, and Joel Emer. 2010. High performance cache replacement using re-reference interval prediction. In *Proceedings of the International Symposium on Computer Architecture (ISCA'10)*.

[46] Jaeheon Jeong and Michel Dubois. 2006. Cache replacement algorithms with nonuniform miss costs. *IEEE Trans. Comput.* 55, 4 (2006), 353–365.

[47] Theodore Johnson and Dennis Shasha. 1994. 2Q: A low overhead high performance buffer management replacement algorithm. In *Proceedings of the 20th International Conference on Very Large Data Bases (VLDB'94)*. Morgan Kaufmann, San Francisco, CA, USA, 439–450.

[48] Nicola Jones et al. 2018. How to stop data centres from gobbling up the world's electricity. *Nature* 561, 7722 (2018), 163–166.

[49] Lucas Joppa. n.d. Made to Measure: Sustainability Commitment Progress and Updates. Retrieved from https://blogs.microsoft.com/blog/2021/07/14/made-to-measure-sustainability-commitment-progress-and-updates/

[50] Ajay Joshi. 2022. CacheLib on ZNS. Retrieved from https://github.com/ajaysjoshi/CacheLib-zns

[51] Jeong-Uk Kang, Jeeseok Hyun, Hyunjoo Maeng, and Sangyeun Cho. 2014. The multi-streamed solid-state drive. In *Proceedings of the 6th USENIX Workshop on Hot Topics in Storage and File Systems (HotStorage'14)*.

[52] Taejin Kim, Duwon Hong, Sangwook Shane Hahn, Myoungjun Chun, Sungjin Lee, Jooyoung Hwang, Jongyoul Lee, and Jihong Kim. 2019. Fully automatic stream management for multi-streamed SSDs using program contexts. In *Proceedings of the 17th USENIX Conference on File and Storage Technologies (FAST'19)*. USENIX Association, Berkeley, CA, 295–308.

[53] Bran Knowles. 2021. ACM TechBrief: Computing and Climate Change. Association for Computing Machinery, New York, NY.

[54] Changman Lee, Dongho Sim, Jooyoung Hwang, and Sangyeun Cho. 2015. F2FS: A new file system for flash storage. In *Proceedings of the USENIX Conference on File and Storage Technologies (FAST'15)*.

[55] Baptiste Lepers, Oana Balmau, Karan Gupta, and Willy Zwaenepoel. 2019. KVell: The design and implementation of a fast persistent key-value store. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles (SOSP'19)*. https://doi.org/10.1145/3341301.3359628

[56] Cheng Li, Philip Shilane, Fred Douglis, Hyong Shim, Stephen Smaldone, and Grant Wallace. 2014. Nitro: A capacity-optimized SSD cache for primary storage. In *Proceedings of the USENIX Annual Technical Conference (USENIX ATC'14)*.

[57] Cheng Li, Philip Shilane, Fred Douglis, and Grant Wallace. 2017. Pannier: Design and analysis of a container-based flash cache for compound objects. *ACM Trans. Stor.* 13, 3 (2017), 24.

[58] Huaicheng Li, Daniel S. Berger, Lisa Hsu, Daniel Ernst, Pantea Zardoshti, Stanko Novakovic, Monish Shah, Samir Rajadnya, Scott Lee, Ishwar Agarwal, et al.. 2023. Pond: CXL-based memory pooling systems for cloud platforms. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS'23)*. Association for Computing Machinery, New York, NY, 574–587. https://doi.org/10.1145/3575693.3578835

[59] Xin Li, Greg Thompson, and Joseph Beer. How Amazon Achieves Near-Real-Time Renewable Energy Plant Monitoring to Optimize Performance Using AWS. n.d. https://aws.amazon.com/blogs/industries/amazon-achieves-near-real-time-renewable-energy-plant-monitoring-to-optimize-performance-using-aws/

[60] Hyeontaek Lim, Bin Fan, David G. Andersen, and Michael Kaminsky. 2011. SILT: A memory-efficient, high-performance key-value store. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles (SOSP'11)*.

[61] Jian Liu, Kefei Wang, and Feng Chen. 2021. TSCache: An efficient flash-based caching scheme for time-series data workloads.

[62] Lanyue Lu, Thanumalayan Sankaranarayana Pillai, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. 2016. WiscKey: Separating keys from values in SSD-conscious storage. In *Proceedings of the USENIX Conference on File and Storage Technologies (FAST'16)*.

[63] Yixin Luo, Saugata Ghose, Yu Cai, Erich F. Haratsch, and Onur Mutlu. 2018. Improving 3D NAND flash memory lifetime by tolerating early retention loss and process variation. *Proc. ACM Meas. Anal. Comput. Syst.* 2, 3, Article 37 (Dec. 2018), 48 pages. https://doi.org/10.1145/3224432

[64] Jialun Lyu, Jaylen Wang, Kali Frost, Chaojie Zhang, Celine Irvene, Esha Choukse, Rodrigo Fonseca, Ricardo Bianchini, Fiodar Kazhamiaka, and Daniel S. Berger. 2023. Myths and misconceptions around reducing carbon embedded in cloud platforms. In *Proceedings of the 2nd Workshop on Sustainable Computer Systems (HotCarbon'23)*. ACM.

[65] Jialun Lyu, Marisa You, Celine Irvene, Mark Jung, Tyler Narmore, Jacob Shapiro, Luke Marshall, Savyasachi Samal, Ioannis Manousakis, Lisa Hsu, et al. 2023. Hyrax:{Fail-in-Place} server operation in cloud platforms. In *Proceedings of the 17th USENIX Symposium on Operating Systems Design and Implementation (OSDI'23)*. 287–304.

[66] Bill Martin, Yoni Shternhell, Mike James, Yeong-Jae Woo, Hyunmo Kang, Anu Murthy, Erich Haratsch, Kwok Kong, Andres Baez, Santosh Kumar, et al. 2022. Nvm Express Technical Proposal 4146 Flexible Data Placement.

[67] Sara McAllister, Benjamin Berg, Julian Tutuncu-Macias, Juncheng Yang, Sathya Gunasekar, Jimmy Lu, Daniel S. Berger, Nathan Beckmann, and Gregory R. Ganger. 2021. Kangaroo: Caching billions of tiny objects on flash. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles (SOSP'21)*.

[68] Sara McAllister, Benjamin Berg, Julian Tutuncu-Macias, Juncheng Yang, Sathya Gunasekar, Jimmy Lu, Daniel S. Berger, Nathan Beckmann, and Gregory R. Ganger. 2022. Kangaroo: Theory and practice of caching billions of tiny objects on flash. *ACM Trans. Stor.* 18, 3 (2022), 1–33.

[69] Jaehong Min, Chenxingyu Zhao, Ming Liu, and Arvind Krishnamurthy. 2023. eZNS: An elastic zoned namespace for commodity ZNS SSDs. In *Proceedings of the 17th USENIX Symposium on Operating Systems Design and Implementation (OSDI'23)*. USENIX Association, Berkeley, CA, 461–477. https://www.usenix.org/conference/osdi23/presentation/min

[70] Christian Monzio Compagnoni, Akira Goda, Alessandro S. Spinelli, Peter Feeley, Andrea L. Lacaita, and Angelo Visconti. 2017. Reviewing the evolution of the NAND flash technology. *Proc. IEEE* 105, 9 (2017), 1609–1633. https://doi.org/10.1109/JPROC.2017.2665781

[71] Melanie Nakagawa. 2022. On the Road to 2030: Our 2022 Environmental Sustainability Report. Retrieved from https://blogs.microsoft.com/on-the-issues/2023/05/10/2022-environmental-sustainability-report/

[72] Jian Ouyang, Shiding Lin, Song Jiang, Zhenyu Hou, Yong Wang, and Yuanzheng Wang. 2014. SDF: Software-defined flash for web-scale internet storage systems. In *Proceedings of the ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS'14)*.

[73] Francisco Pires. 2022. Solidigm Introduces Industry-First PLC NAND for Higher Storage Densities. Retrieved from https://www.tomshardware.com/news/solidigm-plc-nand-ssd

[74] Martin Raab and Angelika Steger. 1998. Balls into Bins: A simple and tight analysis. In *Randomization and Approximation Techniques in Computer Science*, Gerhard Goos, Juris Hartmanis, Jan van Leeuwen, Michael Luby, José D. P. Rolim, and Maria Serna (Eds.). Vol. 1518. Springer, Berlin, 159–170. https://doi.org/10.1007/3-540-49543-6$_1$3. Series Title: Lecture Notes in Computer Science.

[75] Pandian Raju, Rohan Kadekodi, Vijay Chidambaram, and Ittai Abraham. 2017. PebblesDB: Building key-value stores using fragmented log-structured merge trees. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles (SOSP'17)*.

[76] Mendel Rosenblum and John K. Ousterhout. 1991. The design and implementation of a log-structured file system. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles (SOSP'91)*.

[77] Stephen M. Rumble, Ankita Kejriwal, and John Ousterhout. 2014. Log-structured memory for DRAM-based storage. In *Proceedings of the12th USENIX Conference on File and Storage Technologies (FAST'14)*. USENIX Association, Berkeley Clara, CA, 1–16. https://www.usenix.org/conference/fast14/technical-sessions/presentation/rumble

[78] Zhaoyan Shen, Feng Chen, Yichen Jia, and Zili Shao. 2018. Didacache: An integration of device and application for flash-based key-value caching. *ACM Trans. Stor.* 14, 3 (2018), 1–32.

[79] Shigeru Shiratake. 2020. Scaling and performance challenges of future DRAM. In *Proceedings of the IEEE International Memory Workshop (IMW'20)*. 1–3. https://doi.org/10.1109/IMW48823.2020.9108122

[80] Billy Tallis. n.d. 2021 NAND Flash Updates from ISSCC: The Leaning Towers of TLC and QLC. Retrieved from https://www.anandtech.com/show/16491/flash-memory-at-isscc-2021

[81] Billy Tallis. n.d. Micron 3D NAND Status Update. Retrieved from https://www.anandtech.com/show/10028/micron-3d-nand-status-update

[82] Chunqiang Tang, Kenny Yu, Kaushik Veeraraghavan, Jonathan Kaldor, Scott Michelson, Thawan Kooburat, Aravind Anbudurai, Matthew Clark, Kabir Gogia, Long Cheng, et al.. 2020. Twine: A unified cluster management system for shared infrastructure. In *Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI'20)*.

[83] Linpeng Tang, Qi Huang, Wyatt Lloyd, Sanjeev Kumar, and Kai Li. 2015. RIPQ: Advanced photo caching on flash for Facebook. In *Proceedings of the USENIX Conference on File and Storage Technologies (FAST'15)*.

[84] Swamit Tannu and Prashant J. Nair. 2022. The dirty secret of SSDs: Embodied carbon. In *Proceedings of the 2nd Workshop on Sustainable Computer Systems (HotCarbon'22)*.

[85] Amanda Tomlinson and George Porter. 2022. Something old, something new: Extending the life of CPUs in datacenters. In *Proceedings of the 2nd Workshop on Sustainable Computer Systems (HotCarbon'22)*.

[86] Ted Tso. n.d. Aligning Filesystems to an SSD's Erase Block Size. Retrieved from https://tytso.livejournal.com/2009/02/20/

[87] Benny Van Houdt. 2013. A mean field model for a class of garbage collection algorithms in flash-based solid state drives. *ACM SIGMETRICS Perf. Eval. Rev.* 41, 1 (2013), 191–202.

[88] Haitao Wang, Zhanhuai Li, Xiao Zhang, Xiaonan Zhao, Xingsheng Zhao, Weijun Li, and Song Jiang. 2018. OC-Cache: An open-channel SSD based cache for multi-tenant systems. In *Proceedings of the IEEE 37th International Performance Computing and Communications Conference (IPCCC'18)*. IEEE, Los Alamitos, CA, 1–6. https://doi.org/10.1109/PCCC.2018.8711079

[89] Qiuping Wang, Jinhong Li, Patrick P. C. Lee, Tao Ouyang, Chao Shi, and Lilong Huang. 2022. Separating data via block invalidation time inference for write amplification reduction in log-structured storage. In *Proceedings of the 20th USENIX Conference on File and Storage Technologies (FAST'22)*. USENIX Association, Berkeley, CA, 429–444.

[90] Xingbo Wu, Yuehai Xu, Zili Shao, and Song Jiang. 2015. LSM-trie: An LSM-tree-based ultra-large key-value store for small data items. In *Proceedings of the USENIX Annual Technical Conference (ATC'15)*.

[91] Shiqin Yan, Huaicheng Li, Mingzhe Hao, Michael Hao Tong, Swaminathan Sundararaman, Andrew A. Chien, and Haryadi S. Gunawi. 2017. Tiny-tail flash: Near-perfect elimination of garbage collection tail latencies in NAND SSDs. *ACM Trans. Storage* 13, 3, Article 22 (Oct. 2017), 26 pages. https://doi.org/10.1145/3121133

[92] Juncheng Yang, Yao Yue, and K. V. Rashmi. 2021. A large-scale analysis of hundreds of in-memory key-value cache clusters at Twitter. *ACM Trans. Stor.* 17, 3 (2021), 1–35.

[93] Juncheng Yang, Yao Yue, and Rashmi Vinayak. 2021. Segcache: A memory-efficient and scalable in-memory key-value cache for small objects. In *Proceedings of the USENIX Symposium on Networked Systems Design and Implementation (NSDI'21)*.

[94] Pan Yang, Ni Xue, Yuqi Zhang, Yangxu Zhou, Li Sun, Wenwen Chen, Zhonggang Chen, Wei Xia, Junke Li, and Kihyoun Kwon. 2019. Reducing garbage collection overhead in SSD based on workload prediction. In *Proceedings of the 11th USENIX Workshop on Hot Topics in Storage and File Systems (HotStorage'19)*. USENIX Association, Berkeley, CA.

[95] Shao-Peng Yang, Minjae Kim, Sanghyun Nam, Juhyung Park, Jin yong Choi, Eyee Hyun Nam, Eunji Lee, Sungjin Lee, and Bryan S. Kim. 2023. Overcoming the memory wall with CXL-Enabled SSDs. In *Proceedings of the USENIX Annual Technical Conference (ATC'23)*.

[96] Yiying Zhang, Leo Prasath Arulraj, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. 2012. De-indirection for flash-based SSDs with nameless writes. In *Proceedings of the 10th USENIX Conference on File and Storage Technologies (FAST'12)*. USENIX Association, Berkeley, CA, 1.

Received 5 November 2024; revised 5 November 2024; accepted 14 February 2025